Statistics with R programming
Assignment 2

STATISTICS WITH R PROGRAMMING

ASSIGNMENT 2

CO2: *Implement looping constructs, control structures, and recursive functions in R to solve computational problems with efficient program logic.*

Nature of Assignment : Individual
Duration : 1 Week
Total Marks : 50
*Submission Date : **12/11/2026***

Code of Conduct:

*I **RAKSHITH R.R (192324156)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assignment Title: Evaluation and Engineering of Computational Solutions Using Loops, Control Structures, Functions and Recursion in R (Total Marks - 50)

Duration:1week

Scenario — Assessment Design Philosophy

This alternative set follows the reference assignment's evaluation-oriented approach. Students are expected to formulate decision logic, construct challenging test conditions, implement solutions using required R constructs, collect quantitative evidence, identify limitations, compare alternatives, and defend engineering decisions.

The assessment follows: Understand → Predict → Design → Implement → Test → Measure → Analyse → Evaluate → Validate.

Q1 – Engineer a Multi-Factor Wildfire Readiness Alert **8 Marks**

Problem Context

A regional monitoring team combines wind speed, relative humidity, recent rainfall, and vegetation-dryness index to assign readiness levels to monitored zones. A weighted average can conceal a dangerous combination, and threshold-only rules can over-alert when indicators disagree. The team requires a transparent rule-based classification program.

Assessment Challenge

- Design justified rules for LOW, MODERATE, HIGH, and EXTREME readiness using arithmetic, relational, and Boolean operators.
- Use a for() or while() loop and if–else logic to process records, with explicit handling for missing measurements.
- Construct four adversarial datasets: ordinary conditions, conflicting indicators, threshold boundaries, and combined extreme indicators. Predict results before execution and compare with actual outputs.
- Record iterations and condition evaluations; identify where a weighted average alone would give a misleading classification.
- Changed condition: process 25,000 zones and guarantee that very high wind plus very low humidity triggers EXTREME regardless of the average score. Revise and validate the rule hierarchy.

Evidence Expected: Decision rules, R code, four test groups, prediction/observation table, counts, changed-condition tests, and evidence-based evaluation.

Q2 – Design a Scholarship Eligibility and Exception Engine **7 Marks**

Problem Context

A scholarship office processes applications using household-income band, academic average, attendance, required-document status, and a formally approved special-category provision. A high academic average cannot compensate for missing mandatory documents, and an approved provision may waive only the conditions explicitly named in policy.

Assessment Challenge

- Build a rule hierarchy using if–else and relational, arithmetic, AND, OR, and NOT operators.

- Implement a reusable function and use a loop to evaluate application records, returning both status and a reason.
- Test at least six conflicting profiles, including exact income/mark thresholds, missing documents, low attendance, overlapping provisions, and a clearly ineligible case.
- Predict and trace each decision; show at least one result that changes when condition order is altered.
- Introduce a policy change that waives the attendance threshold for one approved category but never waives document verification. Modify the design without duplicating the full decision engine.

Evidence Expected: Priority rules, function, batch processing, six or more adversarial tests, reason codes, condition traces, and before/after policy evaluation.

Q3 – Build a Menu-Driven Online Order Analytics Utility **7 Marks**

Problem Context

An online retailer needs a small R utility to analyse order amounts and quantities. Analysts should be able to select several operations in one session and extend the tool later. Real data may contain missing values, repeated amounts, refunds represented by negative values, and empty subsets.

Assessment Challenge

- Write modular functions for at least five operations selected from total revenue, mean/median order value, range, standard deviation, count of refunds, count above a threshold, and number of missing values.
- Use switch() or an equivalent selection structure and a loop for repeated menu requests and exit.
- Test repeated values, negative/refund amounts, outliers, missing data, one observation, empty input, and invalid menu options.
- Predict outputs and function paths before running the program; compare expected and observed results.
- Add two new operations and evaluate whether the function design avoids unnecessary duplication. Record function calls or menu iterations.

Evidence Expected: Modular functions, menu control, correct operations, edge-case handling, predicted/observed results, counts, and maintainability evaluation.

Q4 – Create an Iterative Package-Dimension Validation Routine 6 Marks

Problem Context

A parcel-sorting system accepts package length in centimetres. For this assignment, valid lengths are 1.0 cm through 120.0 cm inclusive. Sensor records can be blank, non-numeric, outside limits, or repeated due to communication retries. The routine must avoid infinite loops and distinguish correction from rejection.

Assessment Challenge

- Implement a validation/recovery loop using while(), if–else, and Boolean/relational checks.
- Define the permitted interval, retry limit, and final disposition after repeated failures.
- Test at least five sequences: valid first reading, invalid then valid, exact endpoints, alternating invalid/valid records, and repeated missing or non-numeric values.
- Predict final status, attempts, and failures before execution; then measure iterations and condition evaluations.
- Redesign the order of checks to reduce unnecessary evaluations while preserving the same accept/reject outcomes.

Evidence Expected: Correct validation logic, declared limits/retry policy, five sequences, prediction-versus-observation evidence, operation counts, and validation-order analysis.

Q5 – Recursively Analyse a Nested File-and-Folder Structure 7 Marks

Problem Context

A digital archive is represented as nested R lists containing folders and file identifiers. Folder depth varies; some folders are empty, while others contain subfolders several levels deep. The archive team needs to count files, not folder containers.

Assessment Challenge

- Represent an irregular archive structure as nested lists and define what counts as a file leaf.
- Implement a recursive function with explicit base and recursive cases, including safe handling of empty folders.
- Test shallow, deeply nested, unequal-depth, empty-folder, and single-file structures.
- Predict file counts and recursive-call counts, then verify outputs.
- Discuss the effects of depth and branching and compare recursion with an iterative stack-based approach.

Evidence Expected: Nested-list model, correct recursive count, base/recursive cases, five test structures, predicted/observed results, call counts, and comparison.

Q6 – Compare Iterative and Recursive Evaluation of a Geometric Series 7 Marks

Problem Context

A scientific calculation repeatedly adds terms of a geometric series for a selected number of terms. A programmer proposes iterative and recursive implementations. The solution must be mathematically consistent, terminate for valid inputs, and be evaluated as input size grows.

Assessment Challenge

- State the recurrence, initial term, common ratio assumptions, and base cases.
- Implement iterative and recursive functions that compute the same partial sum.
- Test at least four increasing term counts, including zero or the minimum valid count and a boundary case for the chosen ratio.
- Compare outputs, loop iterations/recursive calls, and optional elapsed time using stated conditions.
- Explain recursion depth, numerical/termination considerations, and maintainability; recommend an approach based on evidence rather than timing alone.

Evidence Expected: Mathematical definition, two implementations, four or more test sizes, correctness comparison, call/iteration counts, and reasoned recommendation.

Q7 – Integrated Diagnose–Redesign–Validate Design Review 8 Marks

Problem Context

Choose one implementation from Q1–Q6 and conduct an engineering review after discovering a reproducible limitation. Examples include a misclassified boundary case, invalid-input failure, redundant condition checks, duplicated functions, or unsafe recursive depth. Your redesign must be tied to measured evidence.

Assessment Challenge

- A. Diagnose the limitation (2 marks): identify the root cause and reproduce the defect or bottleneck.
- B. Establish evaluation criteria (1 mark): choose measurable correctness, operation-count, scalability, or maintainability criteria.
- C. Propose and implement a redesign (2 marks): show pseudocode/design logic, implement the change, and explain one trade-off.
- D. Validate and compare (2 marks): test both versions with equivalent inputs and report quantitative results.

- E. Defend the final engineering judgment (1 mark): determine whether the redesign met its objective and identify when the original design may remain useful.

Evidence Expected: Reproducible diagnosis, measurable criteria, redesigned logic, equivalent-condition test evidence, trade-off, and final justified judgment.

Submission Requirements

Submit:

- Assignment report containing evidence for Q1–Q7.
- R source code for all implemented solutions.
- Student-designed datasets and adversarial test cases.
- Raw experimental data and program outputs.
- Prediction-versus-observation comparisons.
- Tables/graphs wherever they strengthen the evaluation.
- Iteration, condition-evaluation, function-call, and recursive-call counts where applicable.
- Pseudocode/design representation for redesigned solutions.

Execution time may be reported as supporting evidence, but conclusions should not rely solely on timing; program-specific measures and logical correctness should form the primary evidence.

Assessment Verification / Viva

Students may be asked to explain and defend decision rules, loop selection, Boolean expressions, switch structure, function decomposition, arguments and return values, recursive base and recursive cases, adversarial test design, measured results, redesign choices, and final engineering judgments.

Overall Assessment Expectation

The assignment assesses computational solutions under non-trivial conditions: Problem Interpretation → Prediction → Design → Implementation → Adversarial Testing → Measurement → Analysis → Evaluation → Validation → Engineering Judgment.

SOLUTION REPORT - ASSIGNMENT 2

Statistics with R Programming: Control-Flow Design, Prediction, Measurement and Evaluation

Every question follows the chain on the assignment sheet: understand the problem, design the logic, **predict before running**, implement, test, measure, analyse and evaluate. Each section therefore shows the design (with a diagram), the predictions, the R code, the captured output, a prediction-versus-observation comparison and a critical evaluation.

Conventions and execution environment

- All code was run in R 4.3.3 with base R only (no add-on packages). Questions 1 to 6 are independent; Question 7 reuses the functions of Question 5, so run Q5 before Q7.
- **Predictions were written into each program before its first run.** Where a prediction was wrong, the comparison table says so and the section explains why. These misses are reported, not hidden.
- Counts (iterations, recursive calls, condition evaluations) are the main evidence. Elapsed times are supporting evidence only: they come from one machine and will differ on another.
- Random data uses fixed seeds, so all counts can be reproduced exactly. The raw datasets are included as CSV files (see Appendix A).

Question map

Q	Task	Marks	Main evidence in this report
1	Multi-factor wildfire readiness alert	8	Rule diagram, 28 adversarial zones, 25,000-zone changed condition
2	Scholarship eligibility and exception engine	7	Priority diagram, 12 profiles, order experiment, one-line policy change
3	Menu-driven order analytics utility	7	63 predicted values, 4 menu sessions, extension and refactor
4	Iterative package-length validation	6	11 sequences, v1 versus v2 counts, 45,000-sequence simulation
5	Recursive file counter for nested folders	7	7 structures, formulas, depth sweep, recursion versus stack
6	Geometric series: iteration versus recursion	7	12 test cases, boundary ratios, depth limit, timing
7	Diagnose, redesign and validate one implementation	8	Two reproduced defects, 5 criteria, 300 random trees, scorecard

Prediction scorecard

Q	Predictions made before running	Matched	Misses and the reason
1	Class of 28 zones under 3 classifiers (84 values)	84 / 84	None
2	Status and reason of 12 profiles under 2 policies (24); reason under a changed order (3)	26 / 27	P9 with documents moved last: I forgot that P9 also has attendance 50, so the first failing check changed
3	9 statistics on 7 datasets (63); 4 menu sessions	63 / 63; 4 / 4	None
4	Status, attempts, failures of 11 sequences (33 values)	33 / 33	None (my hand estimate of the v1/v2 crossover, 0.83, was wrong: measured 0.67)
5	Files, calls and depth of 7 structures (21); 7 complete-tree formulas	26 / 28	Depth of T3 and T4 predicted 6 and 4, observed 5 and 3: I counted one level too many
6	Sum, iterations and calls for 12 cases (36)	34 / 36	n = 1000: recursion fails with a C stack error, so its sum and call count do not exist
7	Failure threshold and quadratic growth expected from Q5/Q6	Confirmed	Exact depth limit depends on calling context (800 safe, 900 fails here)

QUESTION 1 - Multi-Factor Wildfire Readiness Alert (8 Marks)

1.1 Problem understanding and rule design

Each zone reports wind speed (km/h), relative humidity (%), rainfall in the last 72 hours (mm) and a vegetation dryness index (0 to 100). The task is to classify every zone as LOW, MODERATE, HIGH or EXTREME using arithmetic, relational and Boolean operators, handle missing data explicitly, and then add a safety override.

Step	Operators used	Definition
Indicator scores (0-100 danger scale)	arithmetic	wind = min(wind, 100); humidity = 100 - RH; rain = max(0, 100 - 10 x mm); dryness as given
Weighted average	arithmetic	0.25 wind + 0.25 humidity + 0.125 rain + 0.375 dryness. Weights are powers of 1/2, so averages at the cut points are exact in binary arithmetic and boundary tests are not spoiled by rounding
Severe indicator	relational	score >= 80; the count of severe indicators is used so one extreme indicator cannot be averaged away
Class from the average and the count	relational + OR	EXTREME if average >= 75 OR severe >= 3; HIGH if average >= 55 OR severe >= 2; MODERATE if average >= 35 OR severe >= 1; otherwise LOW
Missing data	NOT / any()	any NA gives INCOMPLETE; values outside their range give INVALID; neither is silently turned into a class
Changed condition (v2)	AND	wind >= 80 AND humidity <= 15 gives EXTREME first, whatever the average

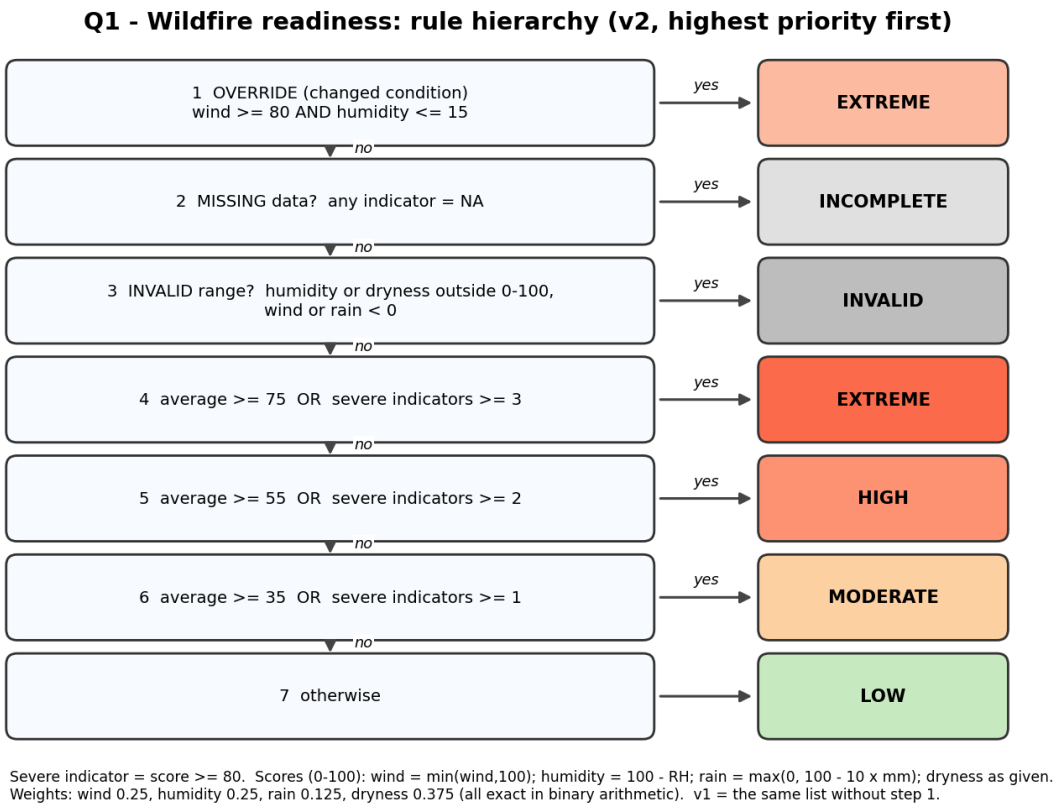


Figure 1: Rule hierarchy of version 2. Version 1 is the same list without step 1

Why the order matters. The override is tested first because it is a safety rule that must win over every other rule, including a missing rain or dryness value. The missing-data and range gates come before the average so that a number is never computed from unusable inputs. The severity counts come after the average so that the average still decides the ordinary cases.

1.2 Adversarial datasets and hand predictions

Group	Purpose	Zones
A ordinary	Typical conditions plus one missing wind value	A1-A7
B conflicting	Indicators disagree: the weighted average hides a dangerous combination (and one zone has missing rain and dryness)	B1-B7
C boundary	Averages of exactly 35, 55 and 75, just below 35 and 55, wind 80 with humidity 15, wind 79.9 with humidity 15.1, humidity 100.5	C1-C8
D combined extreme	All indicators extreme, wind above the 100 cap, and extreme indicators offset by a calm one	D1-D6

Prediction worksheet (inputs, hand-calculated average and severe count, predicted classes)

Zone	Wind	Hum	Rain	Dry	Avg	Severe	Average only	v1	v2
A1	10	70	5	20	23.75	0	LOW	LOW	LOW
A2	30	50	2	50	48.75	1	MODERATE	MODERATE	MODERATE
A3	50	35	0.5	70	66.875	1	HIGH	HIGH	HIGH
A4	70	20	0	90	83.75	3	EXTREME	EXTREME	EXTREME
A5	5	90	12	5	5.625	0	LOW	LOW	LOW
A6	25	60	3	40	40	0	MODERATE	MODERATE	MODERATE
A7	NA	70	5	20	-	-	INCOMPLETE	INCOMPLETE	INCOMPLETE
B1	95	10	10	5	48.125	2	MODERATE	HIGH	EXTREME
B2	5	90	0	100	53.75	2	MODERATE	HIGH	HIGH
B3	90	12	8	10	50.75	2	MODERATE	HIGH	EXTREME
B4	60	80	0	90	66.25	2	HIGH	HIGH	HIGH
B5	100	5	0	0	61.25	3	HIGH	EXTREME	EXTREME
B6	0	100	0	100	50	2	MODERATE	HIGH	HIGH
B7	95	8	NA	NA	-	-	INCOMPLETE	INCOMPLETE	EXTREME
C1	40	60	7	30	35	0	MODERATE	MODERATE	MODERATE
C2	40	60	7	29	34.625	0	LOW	LOW	LOW
C3	60	40	5	50	55	0	HIGH	HIGH	HIGH
C4	60	40	5	49	54.625	0	MODERATE	MODERATE	MODERATE
C5	70	30	2	80	75	2	EXTREME	EXTREME	EXTREME
C6	80	15	5	10	51.25	2	MODERATE	HIGH	EXTREME
C7	79.9	15.1	5	10	51.2	1	MODERATE	MODERATE	MODERATE
C8	40	100.5	7	30	-	-	INVALID	INVALID	INVALID
D1	100	0	0	100	100	4	EXTREME	EXTREME	EXTREME
D2	120	0	0	100	100	4	EXTREME	EXTREME	EXTREME
D3	85	10	0	95	91.875	4	EXTREME	EXTREME	EXTREME
D4	75	12	1	85	83.875	3	EXTREME	EXTREME	EXTREME
D5	100	0	15	0	50	2	MODERATE	HIGH	EXTREME
D6	90	14	9.9	100	81.625	3	EXTREME	EXTREME	EXTREME

Worked examples. **B1** (95, 10, 10, 5): scores 95, 90, 0, 5; average 0.25(95) + 0.25(90) + 0.125(0) + 0.375(5) = 48.125, which the average alone calls MODERATE. Two indicators are severe, so v1 predicts HIGH, and the override makes v2 EXTREME. **C1** (40, 60, 7, 30): scores 40, 40, 30, 30; average exactly 35, so MODERATE because the cut point is inclusive. **C7** (79.9, 15.1, 5, 10): just outside the override, one severe indicator, so MODERATE in both versions.

1.3 Implementation

R code

```
# Q1: Multi-factor wildfire readiness alert
options(width = 85)
set.seed(2026)

# ---- Rule parameters (weights are powers of 1/2 so boundary arithmetic is exact) ----
w   <- c(wind = 0.25, humidity = 0.25, rain = 0.125, dryness = 0.375)
SEV <- 80                    # indicator score at or above this is "severe"
CUT <- c(MODERATE = 35, HIGH = 55, EXTREME = 75)   # average-score cut points
OVR_WIND <- 80; OVR_HUM <- 15 # changed condition: very high wind + very low humidity

cnt <- new.env(); cnt$evals <- 0
chk <- function(cond) { cnt$evals <- cnt$evals + 1; cond }    # counts each condition tested

indicator_scores <- function(wind, hum, rain, dry)           # each on a 0-100 danger scale
  c(wind = min(wind, 100), humidity = 100 - hum,
    rain = max(0, 100 - 10 * rain), dryness = dry)

classify_zone <- function(wind, hum, rain, dry, version = "v2") {
  cnt$evals <- 0
  if (version == "v2") {                    # priority 1: override rule (needs only wind + humidity)
    if (chk(!is.na(wind)) && chk(!is.na(hum)) && chk(wind >= OVR_WIND) && chk(hum <= OVR_HUM))
      return(list(class = "EXTREME", avg = NA_real_, n_sev = NA_integer_, evals = cnt$evals, rule =
"OVERRIDE"))
  }
  if (chk(any(is.na(c(wind, hum, rain, dry))))                        # explicit missing-data handling
    return(list(class = "INCOMPLETE", avg = NA_real_, n_sev = NA_integer_, evals = cnt$evals, rule =
"MISSING"))
  if (chk(wind < 0 || hum < 0 || hum > 100 || rain < 0 || dry < 0 || dry > 100))
    return(list(class = "INVALID", avg = NA_real_, n_sev = NA_integer_, evals = cnt$evals, rule = "RANGE"))
  s <- indicator_scores(wind, hum, rain, dry)
```

```
  avg <- sum(w * s); n_sev <- sum(s >= SEV)
  cl <- if (chk(avg >= CUT["EXTREME"]) || chk(n_sev >= 3)) "EXTREME"
        else if (chk(avg >= CUT["HIGH"]) || chk(n_sev >= 2)) "HIGH"
        else if (chk(avg >= CUT["MODERATE"]) || chk(n_sev >= 1)) "MODERATE"
        else "LOW"
  list(class = cl, avg = avg, n_sev = n_sev, evals = cnt$evals, rule = "HIERARCHY")
}

avg_only <- function(wind, hum, rain, dry) {     # baseline: weighted average alone
  if (any(is.na(c(wind, hum, rain, dry)))) return("INCOMPLETE")
  if (wind < 0 || hum < 0 || hum > 100 || rain < 0 || dry < 0 || dry > 100) return("INVALID")
  a <- sum(w * indicator_scores(wind, hum, rain, dry))
  if (a >= 75) "EXTREME" else if (a >= 55) "HIGH" else if (a >= 35) "MODERATE" else "LOW"
}

process_zones <- function(df, version) {          # for loop over records
  n <- nrow(df); cls <- character(n); ev <- integer(n); avg <- numeric(n); ns <- integer(n); rl <- character(n)
  iterations <- 0
  for (i in seq_len(n)) {
    iterations <- iterations + 1
    r <- classify_zone(df$wind[i], df$hum[i], df$rain[i], df$dry[i], version)
    cls[i] <- r$class; ev[i] <- r$evals; avg[i] <- r$avg; ns[i] <- r$n_sev; rl[i] <- r$rule
  }
  list(data = data.frame(zone = df$zone, class = cls, avg = avg, n_sev = ns, rule = rl, evals = ev,
                         stringsAsFactors = FALSE), iterations = iterations)
}

# ---- Four adversarial datasets (zone, wind km/h, humidity %, rain mm/72h, dryness 0-100) ----
zones <- data.frame(
  zone = c(paste0("A", 1:7), paste0("B", 1:7), paste0("C", 1:8), paste0("D", 1:6)),
  wind = c(10, 30, 50, 70, 5, 25, NA,      95, 5, 90, 60, 100, 0, 95,
           40, 40, 60, 60, 70, 80, 79.9, 40,      100, 120, 85, 75, 100, 90),
  hum  = c(70, 50, 35, 20, 90, 60, 70,     10, 90, 12, 80, 5, 100, 8,
           60, 60, 40, 40, 30, 15, 15.1, 100.5,   0, 0, 10, 12, 0, 14),
  rain = c(5, 2, 0.5, 0, 12, 3, 5,         10, 0, 8, 0, 0, 0, NA,
           7, 7, 5, 5, 2, 5, 5, 7,                 0, 0, 0, 1, 15, 9.9),
  dry  = c(20, 50, 70, 90, 5, 40, 20,      5, 100, 10, 90, 0, 100, NA,
```

```
        30, 29, 50, 49, 80, 10, 10, 30,        100, 100, 95, 85, 0, 100))
zones$group <- c(rep("A ordinary", 7), rep("B conflicting", 7), rep("C boundary", 8), rep("D combined extreme",
6))

# ---- PREDICTIONS written by hand BEFORE running (see report for the arithmetic) ----
pred_v1 <- c("LOW","MODERATE","HIGH","EXTREME","LOW","MODERATE","INCOMPLETE",
          "HIGH","HIGH","HIGH","HIGH","EXTREME","HIGH","INCOMPLETE",
          "MODERATE","LOW","HIGH","MODERATE","EXTREME","HIGH","MODERATE","INVALID",
          "EXTREME","EXTREME","EXTREME","EXTREME","HIGH","EXTREME")
pred_v2 <- c("LOW","MODERATE","HIGH","EXTREME","LOW","MODERATE","INCOMPLETE",
          "EXTREME","HIGH","EXTREME","HIGH","EXTREME","HIGH","EXTREME",

"MODERATE","LOW","HIGH","MODERATE","EXTREME","EXTREME","MODERATE","INVALID",
          "EXTREME","EXTREME","EXTREME","EXTREME","EXTREME","EXTREME")
pred_avg <- c("LOW","MODERATE","HIGH","EXTREME","LOW","MODERATE","INCOMPLETE",
           "MODERATE","MODERATE","MODERATE","HIGH","HIGH","MODERATE","INCOMPLETE",

"MODERATE","LOW","HIGH","MODERATE","EXTREME","MODERATE","MODERATE","INVALID",
           "EXTREME","EXTREME","EXTREME","EXTREME","MODERATE","EXTREME")

r1 <- process_zones(zones, "v1"); r2 <- process_zones(zones, "v2")
obs_avg <- mapply(avg_only, zones$wind, zones$hum, zones$rain, zones$dry)

cat("=== Prediction vs observation: 28 adversarial zones ===\n")
cmp <- data.frame(zone = zones$zone, v1_pred = pred_v1, v1_obs = r1$data$class,
                  v2_pred = pred_v2, v2_obs = r2$data$class, avg_pred = pred_avg, avg_obs = obs_avg)
cmp$ok <- ifelse(cmp$v1_pred == cmp$v1_obs & cmp$v2_pred == cmp$v2_obs & cmp$avg_pred ==
cmp$avg_obs, "yes", "NO")
print(cmp, row.names = FALSE)
cat("\nMatches  v1:", sum(cmp$v1_pred == cmp$v1_obs), "/ 28 | v2:", sum(cmp$v2_pred == cmp$v2_obs),
   "/ 28 | average-only:", sum(cmp$avg_pred == cmp$avg_obs), "/ 28\n")

cat("\n=== Details: average score, severe-indicator count and rule that fired (v2) ===\n")
det <- data.frame(zone = zones$zone, avg = round(r1$data$avg, 3), n_severe = r1$data$n_sev, rule_v2 =
r2$data$rule)
print(det, row.names = FALSE)
```

```
cat("\n=== Iterations and condition evaluations (28 zones) ===\n")
cat("v1: iterations =", r1$iterations, "| condition evaluations =", sum(r1$data$evals), "\n")
cat("v2: iterations =", r2$iterations, "| condition evaluations =", sum(r2$data$evals), "\n")
by_grp <- aggregate(cbind(v1 = r1$data$evals, v2 = r2$data$evals) ~ group, data = zones, FUN = sum)
print(by_grp, row.names = FALSE)

cat("\n=== Where the weighted average alone misleads (average-only vs hierarchy) ===\n")
rank <- c(INCOMPLETE = 0, INVALID = 0, LOW = 1, MODERATE = 2, HIGH = 3, EXTREME = 4)
mis <- cmp$avg_obs != cmp$v2_obs & rank[cmp$avg_obs] < rank[cmp$v2_obs]
print(data.frame(zone = cmp$zone[mis], average_only = cmp$avg_obs[mis], v1 = cmp$v1_obs[mis], v2 =
cmp$v2_obs[mis]), row.names = FALSE)
cat("Zones under-classified by the average alone:", sum(mis), "of 28\n")

# ---- Changed condition: 25,000 zones ----
cat("\n=== CHANGED CONDITION: 25,000 zones ===\n")
n <- 25000
big <- data.frame(zone = sprintf("Z%05d", 1:n),
                  wind = pmax(0, rnorm(n, 35, 22)), hum = pmin(100, pmax(0, rnorm(n, 50, 22))),
                  rain = rexp(n, 1 / 4), dry = pmin(100, pmax(0, rnorm(n, 50, 25))))
inj <- sample(n, 1000)                              # force 1,000 dangerous wind + humidity cases
big$wind[inj] <- runif(1000, 80, 110); big$hum[inj] <- runif(1000, 0, 15)
big$rain[sample(n, 500)] <- NA; big$dry[sample(n, 500)] <- NA       # missing measurements
big$wind[sample(n, 250)] <- NA; big$hum[sample(n, 250)] <- NA
t1 <- system.time(b1 <- process_zones(big, "v1"))[["elapsed"]]
t2 <- system.time(b2 <- process_zones(big, "v2"))[["elapsed"]]
cat("Loop iterations: v1 =", b1$iterations, "| v2 =", b2$iterations, "\n")
cat("Elapsed (s): v1 =", round(t1, 2), "| v2 =", round(t2, 2), "(supporting evidence only)\n")
cat("Total condition evaluations: v1 =", sum(b1$data$evals), "| v2 =", sum(b2$data$evals),
    "| mean per zone: v1 =", round(mean(b1$data$evals), 2), "v2 =", round(mean(b2$data$evals), 2), "\n")
lv <- c("LOW", "MODERATE", "HIGH", "EXTREME", "INCOMPLETE", "INVALID")
dist <- rbind(v1 = table(factor(b1$data$class, lv)), v2 = table(factor(b2$data$class, lv)))
cat("\nClass counts:\n"); print(dist)
trig <- which(!is.na(big$wind) & !is.na(big$hum) & big$wind >= OVR_WIND & big$hum <= OVR_HUM)
cat("\nZones meeting the override condition (wind >= 80 and humidity <= 15):", length(trig), "\n")
cat("  classified EXTREME by v1:", sum(b1$data$class[trig] == "EXTREME"), "\n")
cat("  classified EXTREME by v2:", sum(b2$data$class[trig] == "EXTREME"), "(guarantee requires all)\n")
cat("  of those, with a missing rain or dryness value:", sum(is.na(big$rain[trig]) | is.na(big$dry[trig])), "\n")
```

```
upgraded <- which(b1$data$class != b2$data$class)
cat("Zones whose class changed from v1 to v2:", length(upgraded), "\n")
print(table(from = b1$data$class[upgraded], to = b2$data$class[upgraded]))
cat("Zones downgraded by v2 (rank lower than v1):", sum(rank[b2$data$class] < rank[b1$data$class]), "\n")
nonov <- setdiff(seq_len(n), trig)
cat("Zones outside the override region: identical class in v1 and v2:", all(b1$data$class[nonov] == b2$data$class[nonov]), "\n")

png("fig/fig_q1_alert.png", width = 1800, height = 900, res = 170)
par(mfrow = c(1, 2), mar = c(5, 5, 4, 1))
pal <- c(LOW = "#a1d99b", MODERATE = "#fdd0a2", HIGH = "#fc9272", EXTREME = "#cb181d",
INCOMPLETE = "grey70", INVALID = "grey40")
s <- sample(n, 4000)
plot(big$wind[s], big$hum[s], col = pal[b2$data$class[s]], pch = 16, cex = 0.6, xlab = "Wind speed (km/h)", ylab = "Relative humidity (%)",
    main = "v2 classes; override region boxed", xlim = c(0, 115), ylim = c(0, 100))
rect(OVR_WIND, 0, 115, OVR_HUM, border = "black", lwd = 2, lty = 2)
legend("topright", legend = names(pal)[1:4], col = pal[1:4], pch = 16, cex = 0.7, bty = "n")
barplot(dist[, 1:4], beside = TRUE, col = c("#9ecae1", "#3182bd"), xlab = "Readiness level", ylab = "Number of zones",
       main = "25,000 zones: v1 vs v2")
legend("topright", legend = c("v1 (no override)", "v2 (override)"), fill = c("#9ecae1", "#3182bd"), bty = "n", cex = 0.8)
dev.off()
```

Output (captured from running the code above)

```
=== Prediction vs observation: 28 adversarial zones ===
 zone    v1_pred    v1_obs    v2_pred    v2_obs   avg_pred    avg_obs  ok
   A1       LOW       LOW       LOW       LOW       LOW       LOW yes
   A2  MODERATE  MODERATE  MODERATE  MODERATE  MODERATE  MODERATE yes
   A3      HIGH      HIGH      HIGH      HIGH      HIGH      HIGH yes
   A4   EXTREME   EXTREME   EXTREME   EXTREME   EXTREME   EXTREME yes
   A5       LOW       LOW       LOW       LOW       LOW       LOW yes
   A6  MODERATE  MODERATE  MODERATE  MODERATE  MODERATE  MODERATE yes
   A7 INCOMPLETE INCOMPLETE INCOMPLETE INCOMPLETE INCOMPLETE INCOMPLETE yes
   B1      HIGH      HIGH   EXTREME   EXTREME  MODERATE  MODERATE yes
```

```
  B2      HIGH      HIGH      HIGH      HIGH   MODERATE   MODERATE yes
  B3      HIGH      HIGH    EXTREME    EXTREME   MODERATE   MODERATE yes
  B4      HIGH      HIGH      HIGH      HIGH      HIGH      HIGH yes
  B5    EXTREME    EXTREME    EXTREME    EXTREME      HIGH      HIGH yes
  B6      HIGH      HIGH      HIGH      HIGH   MODERATE   MODERATE yes
  B7 INCOMPLETE INCOMPLETE    EXTREME    EXTREME INCOMPLETE INCOMPLETE yes
  C1   MODERATE   MODERATE   MODERATE   MODERATE   MODERATE   MODERATE yes
  C2       LOW       LOW       LOW       LOW       LOW       LOW yes
  C3      HIGH      HIGH      HIGH      HIGH      HIGH      HIGH yes
  C4   MODERATE   MODERATE   MODERATE   MODERATE   MODERATE   MODERATE yes
  C5    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME yes
  C6      HIGH      HIGH    EXTREME    EXTREME   MODERATE   MODERATE yes
  C7   MODERATE   MODERATE   MODERATE   MODERATE   MODERATE   MODERATE yes
  C8    INVALID    INVALID    INVALID    INVALID    INVALID    INVALID yes
  D1    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME yes
  D2    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME yes
  D3    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME yes
  D4    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME yes
  D5      HIGH      HIGH    EXTREME    EXTREME   MODERATE   MODERATE yes
  D6    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME    EXTREME yes

Matches  v1: 28 / 28 | v2: 28 / 28 | average-only: 28 / 28

=== Details: average score, severe-indicator count and rule that fired (v2) ===
 zone     avg n_severe   rule_v2
   A1  23.750        0 HIERARCHY
   A2  48.750        1 HIERARCHY
   A3  66.875        1 HIERARCHY
   A4  83.750        3 HIERARCHY
   A5   5.625        0 HIERARCHY
   A6  40.000        0 HIERARCHY
   A7      NA       NA   MISSING
   B1  48.125        2  OVERRIDE
   B2  53.750        2 HIERARCHY
   B3  50.750        2  OVERRIDE
   B4  66.250        2 HIERARCHY
   B5  61.250        3  OVERRIDE
```

```
  B6  50.000       2 HIERARCHY
  B7      NA      NA  OVERRIDE
  C1  35.000       0 HIERARCHY
  C2  34.625       0 HIERARCHY
  C3  55.000       0 HIERARCHY
  C4  54.625       0 HIERARCHY
  C5  75.000       2 HIERARCHY
  C6  51.250       2  OVERRIDE
  C7  51.200       1 HIERARCHY
  C8      NA      NA     RANGE
  D1 100.000       4  OVERRIDE
  D2 100.000       4  OVERRIDE
  D3  91.875       4  OVERRIDE
  D4  83.875       3 HIERARCHY
  D5  50.000       2  OVERRIDE
  D6  81.625       3  OVERRIDE

=== Iterations and condition evaluations (28 zones) ===
v1: iterations = 28 | condition evaluations = 139
v2: iterations = 28 | condition evaluations = 190
            group v1 v2
        A ordinary 39 58
     B conflicting 34 42
        C boundary 45 64
 D combined extreme 21 26

=== Where the weighted average alone misleads (average-only vs hierarchy) ===
 zone average_only         v1      v2
  B1     MODERATE       HIGH EXTREME
  B2     MODERATE       HIGH    HIGH
  B3     MODERATE       HIGH EXTREME
  B5         HIGH    EXTREME EXTREME
  B6     MODERATE       HIGH    HIGH
  B7   INCOMPLETE INCOMPLETE EXTREME
  C6     MODERATE       HIGH EXTREME
  D5     MODERATE       HIGH EXTREME
Zones under-classified by the average alone: 8 of 28
```

=== CHANGED CONDITION: 25,000 zones ===

Loop iterations: v1 = 25000 | v2 = 25000

Elapsed (s): v1 = 0.24 | v2 = 0.28 (supporting evidence only)

Total condition evaluations: v1 = 152236 | v2 = 224093 | mean per zone: v1 = 6.09 v2 = 8.96

Class counts:

	LOW	MODERATE	HIGH	EXTREME	INCOMPLETE	INVALID
v1	2496	13083	7021	936	1464	0
v2	2496	13083	6661	1331	1429	0

Zones meeting the override condition (wind >= 80 and humidity <= 15): 1011

classified EXTREME by v1: 616

classified EXTREME by v2: 1011 (guarantee requires all)

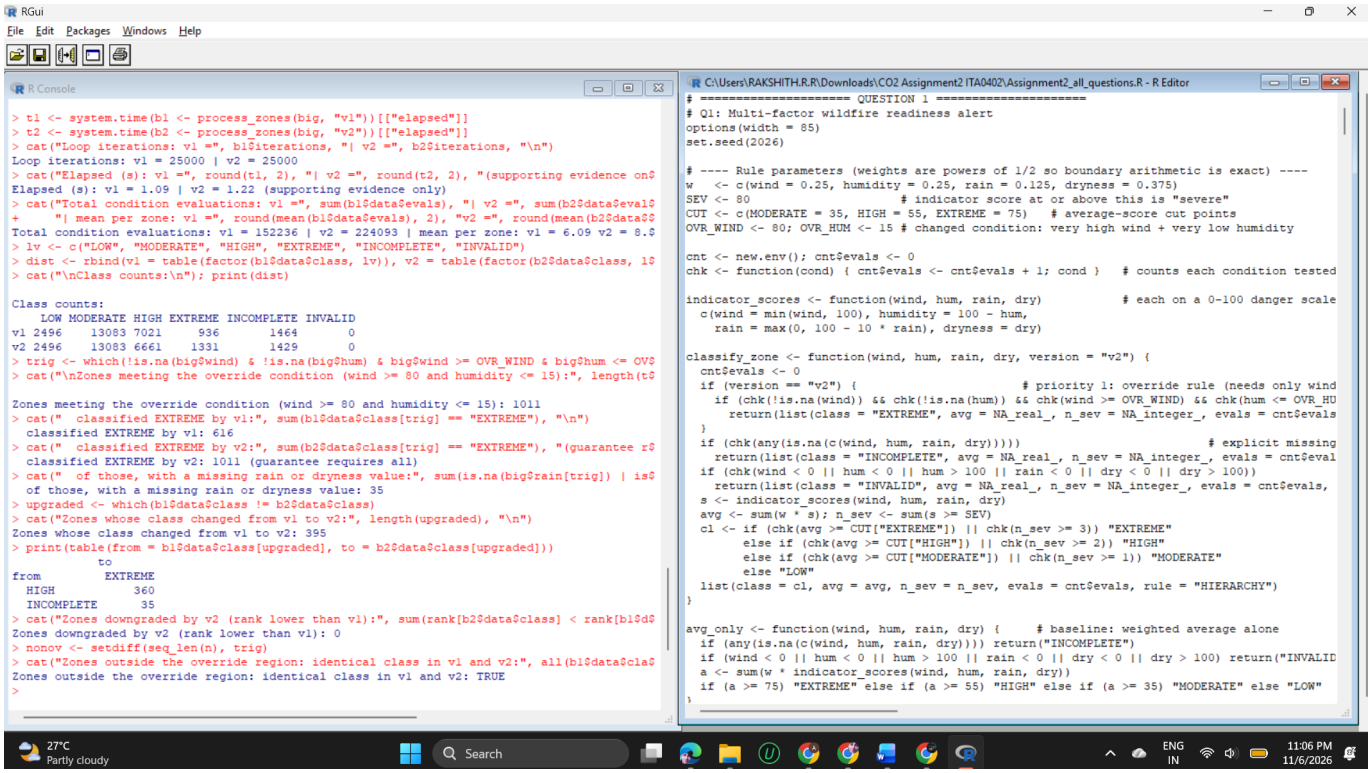
of those, with a missing rain or dryness value: 35

Zones whose class changed from v1 to v2: 395

from \ to	EXTREME
HIGH	360
INCOMPLETE	35

Zones downgraded by v2 (rank lower than v1): 0

Zones outside the override region: identical class in v1 and v2: TRUE



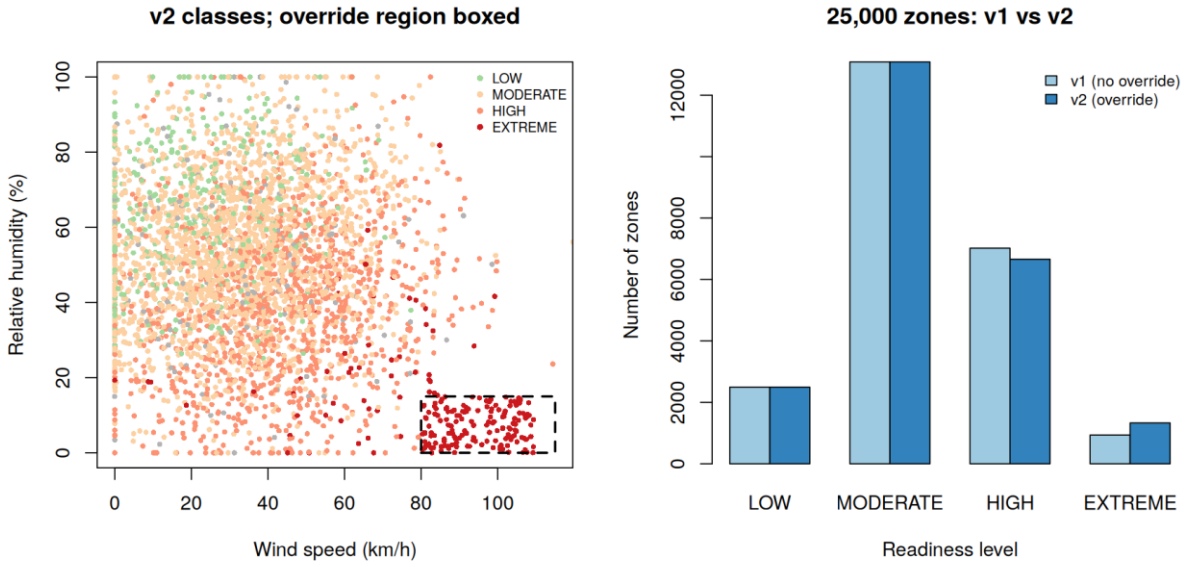


Figure 2: Left: 4,000 sampled zones coloured by their v2 class with the override region boxed. Right: class counts for 25,000 zones, v1 against v2

1.4 Prediction versus observation

- **28 of 28 predictions matched for each of the three classifiers** (84 of 84), including every boundary: average 35 gave MODERATE, 34.625 gave LOW, average 55 gave HIGH, 54.625 gave MODERATE, average 75 gave EXTREME, wind 80 with humidity 15 triggered the override, and 79.9 with 15.1 did not.

- **Loop and condition counts (28 zones):** 28 iterations in each version. Condition evaluations were 139 for v1 and 190 for v2. The override adds up to four tests per zone before the normal hierarchy starts.
- **Where the weighted average alone misleads:** 8 of 28 zones were classified lower by the average than by the hierarchy (B1, B2, B3, B5, B6, B7, C6, D5). For example D5 (wind 100, humidity 0, rain 15, dryness 0) averages exactly 50 because wet, green vegetation pulls it down, although wind and humidity are both at the worst possible score.
- **The hierarchy also guards against the opposite error.** B2 and B6 have two severe indicators (very dry vegetation, no rain) but low wind and high humidity; a rule such as 'any severe indicator means EXTREME' would over-alert, whereas the hierarchy gives HIGH.

1.5 Changed condition at scale (25,000 zones)

- **1,011** zones met the override condition. v1 classified only **616** of them EXTREME; v2 classified **all 1,011**.
- 395 zones changed class from v1 to v2: 360 went from HIGH to EXTREME and 35 from INCOMPLETE to EXTREME. These 35 had a missing rain or dryness value, so the guarantee holds even with incomplete data.
- No zone was downgraded, and every zone outside the override region kept exactly the same class in v1 and v2.
- Cost: mean condition evaluations per zone rose from 6.09 to 8.96 (about 47% more); the loop time moved from 0.24 s to 0.28 s (supporting evidence only).

1.6 Evaluation and limitations

- **Strength:** the weighted average gives a smooth ranking, the severity count protects against averaging away a dangerous indicator, and the override encodes the one case that must never be missed.
- **Limitation 1:** the override is tested before the range check, so an impossible humidity such as -5 would raise an EXTREME alert. A production version should add a range test inside the override.
- **Limitation 2:** the weights, cut points and the severity score of 80 are chosen for the exercise and are not calibrated on real fire data.
- **Limitation 3:** a faulty wind or humidity sensor can now trigger EXTREME on its own. This is a deliberate safety-first trade-off: a false alarm is cheaper than a missed one.

QUESTION 2 - Scholarship Eligibility and Exception Engine (7 Marks)

2.1 Problem understanding and design

A scholarship decision depends on a household-income band (1 lowest to 5), the academic average, attendance, document status and any approved special provision. The design rule is that **priority and waiver logic live in a policy table, and the decision function only reads it.** Changing the policy is then a data edit, not a code change.

Condition	**Rule (policy v1)**	**Reason code on failure**	**May a provision waive it?**
Data validity	no missing values; average and attendance within 0-100	REVIEW / DATA_INVALID	No
Documents	status is VERIFIED	DOC_MISSING	Never (safeguard in the engine)
Income	band <= 3	INCOME_ABOVE_LIMIT	SINGLE_PARENT
Academic average	>= 60	LOW_AVERAGE	SPORTS
Attendance	>= 75	LOW_ATTENDANCE	None in v1; DISABILITY in v2

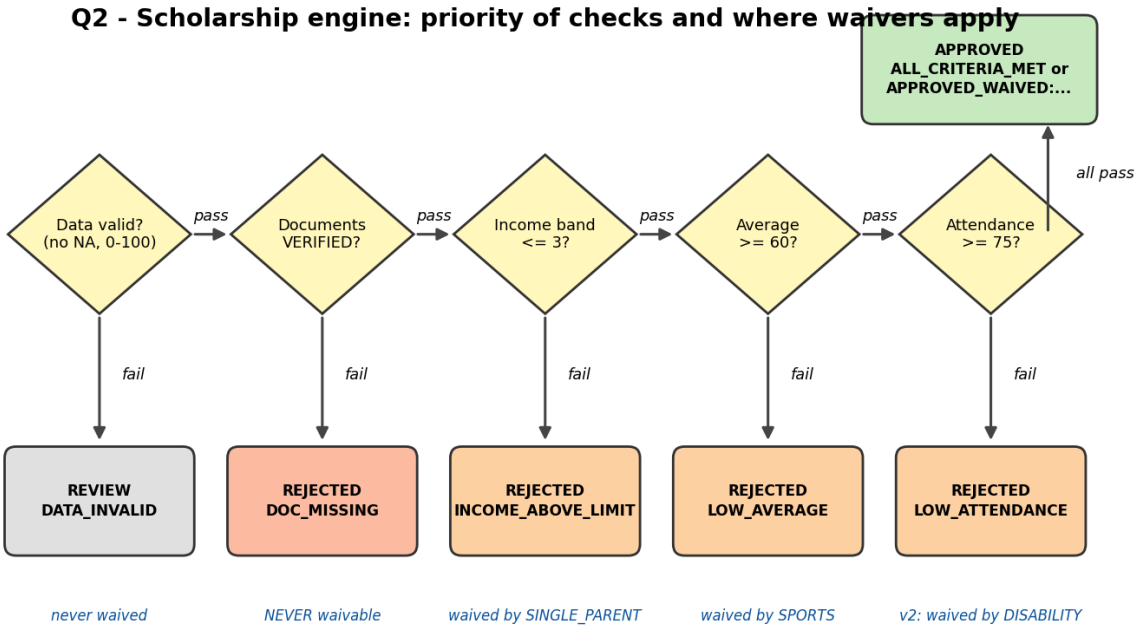


Figure 3: Order of the checks. A failed check is waived only by an APPROVED provision that the policy names for it

- **First failure decides:** checks run in the order documents, income, average, attendance, so the reason code is the first failed condition.
- **Waivers are specific:** a provision waives only the condition named for it; an unlisted provision waives nothing; a PENDING provision waives nothing; overlapping provisions each act on their own condition.
- **NOT logic:** the waivable set is the named conditions minus never_waivable, so documents can never be waived even if someone types it into the table by mistake (tested below).
- **Arithmetic:** thresholds are compared with relational operators; the loop over application records is a for loop.

2.2 Test profiles and predictions

ID	What the profile tests
P1	Every value exactly on its threshold (band 3, average 60.0, attendance 75.0)
P2	Average 59.99, just below the threshold
P3	Excellent academics (98) but documents missing, with an approved SPORTS provision
P4	Attendance 74.9, just below the threshold
P5	Overlapping provisions SPORTS and SINGLE_PARENT: band 4 and average 55 both need a waiver
P6	Clearly ineligible on every condition
P7	SPORTS provision still PENDING, average 55

P8	Approved DISABILITY provision with attendance 60
P9	Approved DISABILITY provision, attendance 50, documents missing
P10	Missing academic average
P11	SPORTS and DISABILITY together: average 55, attendance 60
P12	Income band 4, no provision

2.3 Implementation

R code

```
# Q2: Scholarship eligibility and exception engine
options(width = 85)

# ---- Policy as DATA (the engine below never changes when the policy changes) ----
policy_v1 <- list(
  min_avg = 60, min_att = 75, max_band = 3,
  waivers = list(SPORTS = "average", SINGLE_PARENT = "income"),    # provision -> the ONLY condition it waives
  never_waivable = "documents",
  order = c("documents", "income", "average", "attendance"))      # priority order of the checks

check_defs <- list(                                  # relational operators on the application record
  documents  = function(a, p) a$docs == "VERIFIED",
  income     = function(a, p) a$income_band <= p$max_band,
  average    = function(a, p) a$avg >= p$min_avg,
  attendance = function(a, p) a$att >= p$min_att)
fail_code <- c(documents = "DOC_MISSING", income = "INCOME_ABOVE_LIMIT",
               average = "LOW_AVERAGE", attendance = "LOW_ATTENDANCE")

evaluate_application <- function(app, policy) {
  trace <- character()
  bad <- is.na(app$avg) || is.na(app$att) || is.na(app$income_band) || is.na(app$docs) ||
         app$avg < 0 || app$avg > 100 || app$att < 0 || app$att > 100
  if (bad) return(list(status = "REVIEW", reason = "DATA_INVALID", trace = "data:INVALID"))
  provs <- if (app$prov_status == "APPROVED" && nzchar(app$prov)) strsplit(app$prov, ";")[[1]] else character()
  waived <- unique(unlist(policy$waivers[intersect(provs, names(policy$waivers))]))
```

```
  waived <- setdiff(waived, policy$never_waivable)              # NOT/AND logic: documents can never be waived
  used <- character()
  for (ck in policy$order) {
    passed <- check_defs[[ck]](app, policy)
    if (!passed && (ck %in% waived)) { trace <- c(trace, paste0(ck, ":WAIVED")); used <- c(used, ck); next }
    if (!passed) {
      trace <- c(trace, paste0(ck, ":FAIL"))
      return(list(status = "REJECTED", reason = unname(fail_code[ck]), trace = paste(trace, collapse = " | ")))
    }
    trace <- c(trace, paste0(ck, ":PASS"))
  }
  list(status = "APPROVED",
       reason = if (length(used) == 0) "ALL_CRITERIA_MET" else paste0("APPROVED_WAIVED:", paste(used,
collapse = "+")),
       trace = paste(trace, collapse = " | "))
}

evaluate_all <- function(apps, policy) {                      # loop over application records
  n <- nrow(apps); st <- re <- tr <- character(n)
  for (i in seq_len(n)) {
    r <- evaluate_application(as.list(apps[i, ]), policy)
    st[i] <- r$status; re[i] <- r$reason; tr[i] <- r$trace
  }
  data.frame(id = apps$id, status = st, reason = re, trace = tr, stringsAsFactors = FALSE)
}

# ---- Twelve adversarial profiles ----
apps <- data.frame(stringsAsFactors = FALSE,
  id = paste0("P", 1:12),
  income_band = c(3, 2, 1, 2, 4, 5, 2, 2, 2, NA, 2, 4),
  avg    = c(60.0, 59.99, 98, 80, 55, 35, 55, 80, 85, 80, 55, 70),
  att    = c(75.0, 90, 95, 74.9, 80, 40, 85, 60, 50, 80, 60, 80),
  docs    = c("VERIFIED", "VERIFIED", "MISSING", "VERIFIED", "VERIFIED", "MISSING", "VERIFIED",
"VERIFIED", "MISSING", "VERIFIED", "VERIFIED", "VERIFIED"),
  prov        = c("", "", "SPORTS", "", "SPORTS;SINGLE_PARENT", "", "SPORTS", "DISABILITY",
"DISABILITY", "", "SPORTS;DISABILITY", ""),
```

```
 prov_status = c("NONE", "NONE", "APPROVED", "NONE", "APPROVED", "NONE", "PENDING",
"APPROVED", "APPROVED", "NONE", "APPROVED", "NONE"))
apps$income_band[10] <- 3; apps$avg[10] <- NA            # P10: missing academic average
cat("--- Applications ---\n"); print(apps, row.names = FALSE)

# ---- PREDICTIONS (hand-traced before execution) ----
pred_v1 <- data.frame(id = apps$id, stringsAsFactors = FALSE,
 status                                                                                =
c("APPROVED","REJECTED","REJECTED","REJECTED","APPROVED","REJECTED","REJECTED","REJ
ECTED","REJECTED","REVIEW","REJECTED","REJECTED"),
 reason                                                                                =
c("ALL_CRITERIA_MET","LOW_AVERAGE","DOC_MISSING","LOW_ATTENDANCE","APPROVED_
WAIVED:income+average","DOC_MISSING",

"LOW_AVERAGE","LOW_ATTENDANCE","DOC_MISSING","DATA_INVALID","LOW_ATTENDANCE
","INCOME_ABOVE_LIMIT"))
pred_v2 <- pred_v1
pred_v2[8, c("status", "reason")]  <- c("APPROVED", "APPROVED_WAIVED:attendance")
pred_v2[11, c("status", "reason")] <- c("APPROVED", "APPROVED_WAIVED:average+attendance")

v1 <- evaluate_all(apps, policy_v1)
cat("\n=== Baseline policy v1: prediction vs observation ===\n")
cmp1 <- data.frame(id = v1$id, predicted = paste(pred_v1$status, pred_v1$reason, sep = "/"),
                   observed = paste(v1$status, v1$reason, sep = "/"),
                   ok = ifelse(pred_v1$status == v1$status & pred_v1$reason == v1$reason, "yes", "NO"))
cmp1$predicted <- sub("APPROVED_WAIVED:", "WAIVED:", cmp1$predicted); cmp1$observed <-
sub("APPROVED_WAIVED:", "WAIVED:", cmp1$observed)
print(cmp1, row.names = FALSE)
cat("Matches:", sum(cmp1$ok == "yes"), "/ 12\n")

cat("\n=== Condition traces (v1) ===\n")
for (i in seq_len(nrow(v1))) cat(sprintf("%-4s %-9s %-30s %s\n", v1$id[i], v1$status[i], v1$reason[i], v1$trace[i]))
cat("Conditions evaluated per profile:", paste(sapply(strsplit(v1$trace, " \\| "), length), collapse = " "), "\n")

# ---- Effect of CONDITION ORDER ----
cat("\n=== Condition-order experiment ===\n")
pol_rev <- policy_v1; pol_rev$order <- c("attendance", "average", "income", "documents")
```

```
ord_tab <- data.frame(id = c("P3", "P6", "P9"),
  docs_first = evaluate_all(apps[c(3, 6, 9), ], policy_v1)$reason,
  docs_last  = evaluate_all(apps[c(3, 6, 9), ], pol_rev)$reason,
  steps_docs_first = sapply(strsplit(evaluate_all(apps[c(3, 6, 9), ], policy_v1)$trace, " \\| "), length),
  steps_docs_last  = sapply(strsplit(evaluate_all(apps[c(3, 6, 9), ], pol_rev)$trace, " \\| "), length))
ord_tab$predicted_docs_last <- c("DOC_MISSING", "LOW_ATTENDANCE", "DOC_MISSING")    # written
before running
ord_tab$prediction_ok <- ifelse(ord_tab$predicted_docs_last == ord_tab$docs_last, "yes", "NO")
print(ord_tab[, c("id", "docs_first", "docs_last", "predicted_docs_last", "prediction_ok")], row.names = FALSE)
cat("Steps (conditions evaluated): docs first =", paste(ord_tab$steps_docs_first, collapse = " "), "| docs last =",
paste(ord_tab$steps_docs_last, collapse = " "), "\n")

naive_provision_first <- function(app, policy) {       # FLAWED ordering: an approved provision short-circuits
everything
  if (app$prov_status == "APPROVED" && nzchar(app$prov)) return(list(status = "APPROVED", reason =
"PROVISION_APPROVED"))
  evaluate_application(app, policy)
}
nv <- sapply(seq_len(nrow(apps)), function(i) naive_provision_first(as.list(apps[i, ]), policy_v1)$status)
chg <- which(nv != v1$status)
cat("\nFlawed 'provision first' ordering changes the STATUS of", length(chg), "profiles:\n")
print(data.frame(id = apps$id[chg], correct = v1$status[chg], provision_first = nv[chg]), row.names = FALSE)

# ---- POLICY CHANGE: waive attendance for DISABILITY only; documents never waived ----
policy_v2 <- policy_v1
policy_v2$waivers$DISABILITY <- "attendance"            # the ONLY edit: one line of policy data
v2 <- evaluate_all(apps, policy_v2)
cat("\n=== Policy v2 (DISABILITY waives attendance): prediction vs observation ===\n")
cmp2 <- data.frame(id = v2$id, predicted = paste(pred_v2$status, pred_v2$reason, sep = "/"),
                   observed = paste(v2$status, v2$reason, sep = "/"),
                   ok = ifelse(pred_v2$status == v2$status & pred_v2$reason == v2$reason, "yes", "NO"))
cmp2$predicted <- sub("APPROVED_WAIVED:", "WAIVED:", cmp2$predicted); cmp2$observed <-
sub("APPROVED_WAIVED:", "WAIVED:", cmp2$observed)
print(cmp2, row.names = FALSE)
cat("Matches:", sum(cmp2$ok == "yes"), "/ 12\n")
cat("\n=== Before / after the policy change ===\n")
```

```
ba <- data.frame(id = apps$id, before = sub("APPROVED_WAIVED:", "WAIVED:", v1$reason), after = sub("APPROVED_WAIVED:", "WAIVED:", v2$reason))
ba$changed <- ifelse(v1$status != v2$status | v1$reason != v2$reason, "CHANGED", "")
print(ba, row.names = FALSE)
cat("Profiles changed:", sum(ba$changed == "CHANGED"), "of 12 | approvals before:", sum(v1$status == "APPROVED"),
    "after:", sum(v2$status == "APPROVED"), "\n")
cat("Edits needed: engine functions changed = 0; policy lines added = 1\n")

cat("\n=== Safeguard: even a mistaken policy entry cannot waive documents ===\n")
policy_bad <- policy_v2; policy_bad$waivers$DISABILITY <- c("attendance", "documents")
sg <- evaluate_all(apps[c(9), ], policy_bad)
cat("P9 (DISABILITY, documents MISSING) under the mistaken policy ->", sg$status, sg$reason, "\n")

png("fig/fig_q2_policy.png", width = 1700, height = 850, res = 170)
par(mfrow = c(1, 2), mar = c(5, 5, 4, 1))
tb <- rbind(v1 = table(factor(v1$status, c("APPROVED", "REJECTED", "REVIEW"))), v2 = table(factor(v2$status, c("APPROVED", "REJECTED", "REVIEW"))))
barplot(tb, beside = TRUE, col = c("#9ecae1", "#3182bd"), ylab = "Number of profiles", xlab = "Final status", main = "Outcomes before and after the change")
legend("topleft", legend = c("policy v1", "policy v2"), fill = c("#9ecae1", "#3182bd"), bty = "n", cex = 0.8)
len1 <- sapply(strsplit(v1$trace, " \\| "), length)
barplot(len1, names.arg = apps$id, col = "#a1d99b", ylab = "Conditions evaluated (same in v1 and v2)", xlab = "Application profile", main = "Condition trace length", las = 2, cex.names = 0.8)
dev.off()
```

Output (captured from running the code above)

```
--- Applications ---
 id income_band   avg  att     docs              prov prov_status
 P1           3 60.00 75.0 VERIFIED                      NONE
 P2           2 59.99 90.0 VERIFIED                      NONE
 P3           1 98.00 95.0  MISSING             SPORTS    APPROVED
 P4           2 80.00 74.9 VERIFIED                      NONE
 P5           4 55.00 80.0 VERIFIED SPORTS;SINGLE_PARENT    APPROVED
 P6           5 35.00 40.0  MISSING                     NONE
 P7           2 55.00 85.0 VERIFIED             SPORTS     PENDING
```

```
 P8          2 80.00 60.0 VERIFIED          DISABILITY    APPROVED
 P9          2 85.00 50.0  MISSING          DISABILITY    APPROVED
P10          3    NA 80.0 VERIFIED                        NONE
P11          2 55.00 60.0 VERIFIED    SPORTS;DISABILITY    APPROVED
P12          4 70.00 80.0 VERIFIED                        NONE

=== Baseline policy v1: prediction vs observation ===
 id                  predicted                  observed  ok
 P1     APPROVED/ALL_CRITERIA_MET     APPROVED/ALL_CRITERIA_MET yes
 P2         REJECTED/LOW_AVERAGE          REJECTED/LOW_AVERAGE yes
 P3         REJECTED/DOC_MISSING          REJECTED/DOC_MISSING yes
 P4       REJECTED/LOW_ATTENDANCE       REJECTED/LOW_ATTENDANCE yes
 P5 APPROVED/WAIVED:income+average APPROVED/WAIVED:income+average yes
 P6         REJECTED/DOC_MISSING          REJECTED/DOC_MISSING yes
 P7         REJECTED/LOW_AVERAGE          REJECTED/LOW_AVERAGE yes
 P8       REJECTED/LOW_ATTENDANCE       REJECTED/LOW_ATTENDANCE yes
 P9         REJECTED/DOC_MISSING          REJECTED/DOC_MISSING yes
P10          REVIEW/DATA_INVALID           REVIEW/DATA_INVALID yes
P11       REJECTED/LOW_ATTENDANCE       REJECTED/LOW_ATTENDANCE yes
P12    REJECTED/INCOME_ABOVE_LIMIT    REJECTED/INCOME_ABOVE_LIMIT yes
Matches: 12 / 12

=== Condition traces (v1) ===
P1    APPROVED    ALL_CRITERIA_MET                    documents:PASS | income:PASS | average:PASS |
attendance:PASS
P2   REJECTED  LOW_AVERAGE                documents:PASS | income:PASS | average:FAIL
P3   REJECTED  DOC_MISSING                documents:FAIL
P4    REJECTED   LOW_ATTENDANCE                    documents:PASS | income:PASS | average:PASS |
attendance:FAIL
P5        APPROVED      APPROVED_WAIVED:income+average    documents:PASS   |   income:WAIVED   |
average:WAIVED | attendance:PASS
P6   REJECTED  DOC_MISSING                documents:FAIL
P7   REJECTED  LOW_AVERAGE                documents:PASS | income:PASS | average:FAIL
P8    REJECTED   LOW_ATTENDANCE                    documents:PASS | income:PASS | average:PASS |
attendance:FAIL
P9   REJECTED  DOC_MISSING                documents:FAIL
P10  REVIEW    DATA_INVALID                data:INVALID
```

```
P11   REJECTED   LOW_ATTENDANCE                    documents:PASS | income:PASS | average:WAIVED |
attendance:FAIL
P12  REJECTED  INCOME_ABOVE_LIMIT            documents:PASS | income:FAIL
Conditions evaluated per profile: 4 3 1 4 4 1 3 4 1 1 4 2

=== Condition-order experiment ===
 id  docs_first      docs_last predicted_docs_last prediction_ok
 P3 DOC_MISSING    DOC_MISSING        DOC_MISSING          yes
 P6 DOC_MISSING LOW_ATTENDANCE      LOW_ATTENDANCE          yes
 P9 DOC_MISSING LOW_ATTENDANCE        DOC_MISSING           NO
Steps (conditions evaluated): docs first = 1 1 1 | docs last = 4 1 1

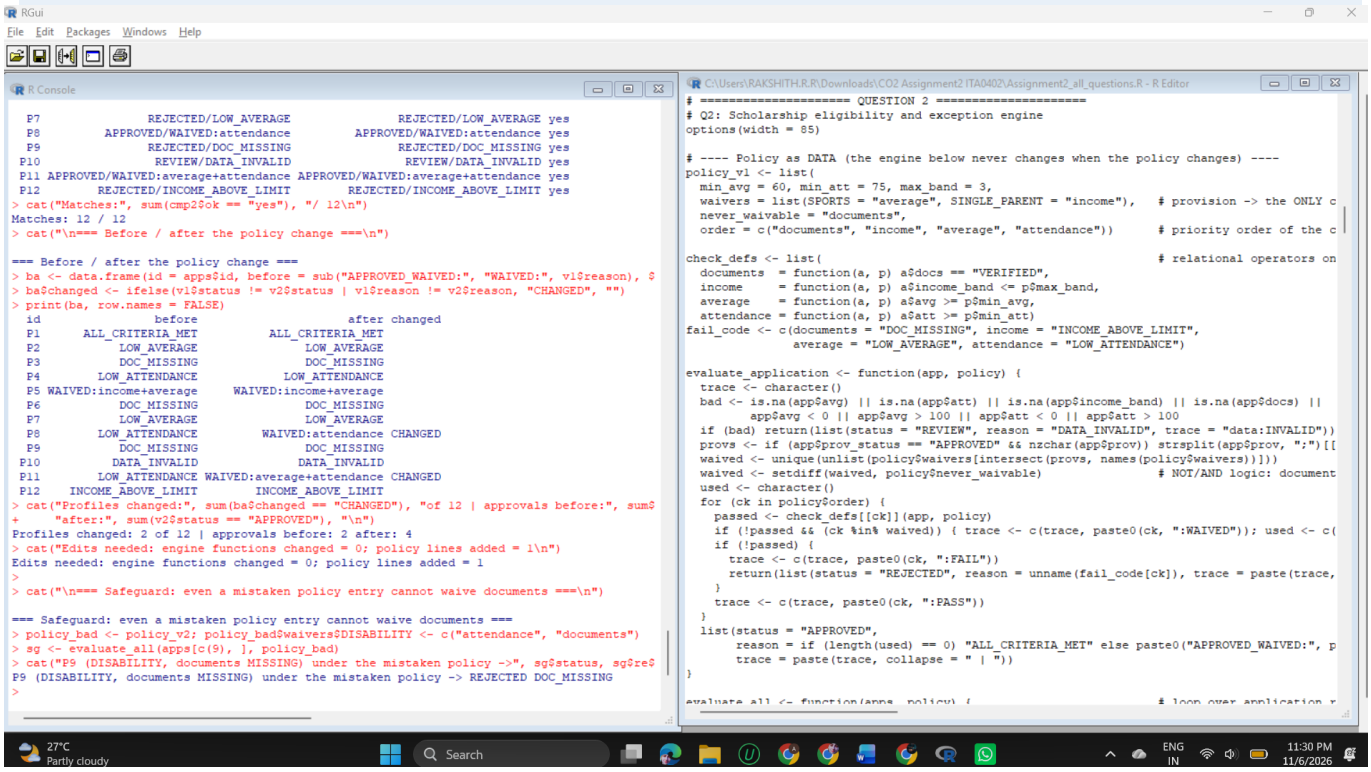
Flawed 'provision first' ordering changes the STATUS of 4 profiles:
  id  correct provision_first
  P3 REJECTED        APPROVED
  P8 REJECTED        APPROVED
  P9 REJECTED        APPROVED
 P11 REJECTED        APPROVED

=== Policy v2 (DISABILITY waives attendance): prediction vs observation ===
  id                      predicted                       observed  ok
  P1         APPROVED/ALL_CRITERIA_MET         APPROVED/ALL_CRITERIA_MET yes
  P2              REJECTED/LOW_AVERAGE              REJECTED/LOW_AVERAGE yes
  P3              REJECTED/DOC_MISSING             REJECTED/DOC_MISSING yes
  P4           REJECTED/LOW_ATTENDANCE           REJECTED/LOW_ATTENDANCE yes
  P5     APPROVED/WAIVED:income+average     APPROVED/WAIVED:income+average yes
  P6              REJECTED/DOC_MISSING             REJECTED/DOC_MISSING yes
  P7              REJECTED/LOW_AVERAGE              REJECTED/LOW_AVERAGE yes
  P8        APPROVED/WAIVED:attendance        APPROVED/WAIVED:attendance yes
  P9              REJECTED/DOC_MISSING             REJECTED/DOC_MISSING yes
 P10               REVIEW/DATA_INVALID               REVIEW/DATA_INVALID yes
 P11 APPROVED/WAIVED:average+attendance APPROVED/WAIVED:average+attendance yes
 P12       REJECTED/INCOME_ABOVE_LIMIT       REJECTED/INCOME_ABOVE_LIMIT yes
Matches: 12 / 12

=== Before / after the policy change ===
  id              before                   after changed
```

```
 P1     ALL_CRITERIA_MET        ALL_CRITERIA_MET
 P2         LOW_AVERAGE            LOW_AVERAGE
 P3         DOC_MISSING            DOC_MISSING
 P4      LOW_ATTENDANCE         LOW_ATTENDANCE
 P5 WAIVED:income+average    WAIVED:income+average
 P6         DOC_MISSING            DOC_MISSING
 P7         LOW_AVERAGE            LOW_AVERAGE
 P8      LOW_ATTENDANCE        WAIVED:attendance CHANGED
 P9         DOC_MISSING            DOC_MISSING
P10         DATA_INVALID           DATA_INVALID
P11      LOW_ATTENDANCE WAIVED:average+attendance CHANGED
P12   INCOME_ABOVE_LIMIT       INCOME_ABOVE_LIMIT
Profiles changed: 2 of 12 | approvals before: 2 after: 4
Edits needed: engine functions changed = 0; policy lines added = 1

=== Safeguard: even a mistaken policy entry cannot waive documents ===
P9 (DISABILITY, documents MISSING) under the mistaken policy -> REJECTED DOC_MISSING
```



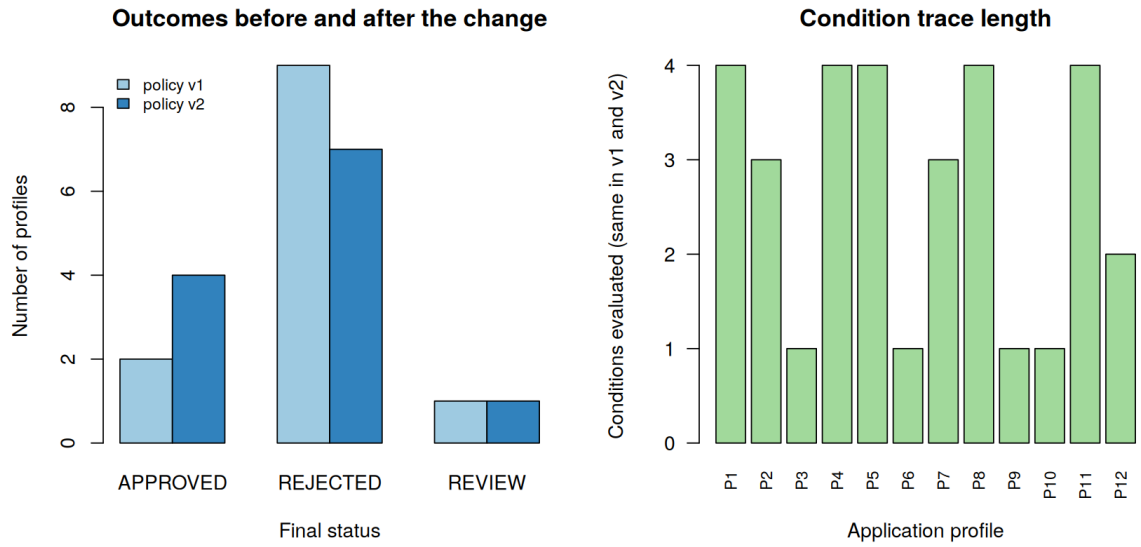


Figure 4: Left: outcomes before and after the policy change. Right: number of conditions evaluated per profile

2.4 Analysis

- **Prediction versus observation:** 12 of 12 for policy v1 and 12 of 12 for policy v2 (status and reason code both matched). Exact thresholds behaved as designed: P1 was approved and P2 and P4 were rejected by 0.01 and 0.1 respectively.
- **Condition traces:** each decision lists the checks in order (for example documents:PASS | income:PASS | average:WAIVED | attendance:FAIL). Evaluation stops at the first unwaived failure, so rejected profiles evaluate between 1 and 4 conditions.
- **Effect of condition order (one prediction missed):** moving documents last changed the **reason code** for P6 (DOC_MISSING became LOW_ATTENDANCE) and for P9 (DOC_MISSING became LOW_ATTENDANCE), and made P3 evaluate 4 conditions instead of 1. I predicted that P9 would keep DOC_MISSING; I had overlooked its attendance of 50, so my prediction was wrong and the observation is correct. In the engine the order never changes the final status, because every condition must pass.
- **A badly ordered design does change the status:** the flawed rule 'an approved provision approves the application at once' changed the status of 4 profiles (P3, P8, P9 and P11) from REJECTED to APPROVED, although two of them have missing documents and the other two fail the attendance rule. This is why provisions are applied as waivers on named conditions after the mandatory checks, not as a shortcut.
- **Policy change:** adding DISABILITY = "attendance" to the waiver table changed 2 of 12 profiles (P8 and P11) and raised approvals from 2 to 4. Engine functions changed: 0. Policy lines added: 1. P9 stayed REJECTED because its documents are missing.
- **Safeguard test:** a mistaken policy entry that lists documents as waivable for DISABILITY still left P9 REJECTED with DOC_MISSING.

2.5 Evaluation and limitations

- Only the **first** failed condition is reported. This is simple and fast, but an applicant who fails two conditions learns about them one at a time. A complete report would evaluate all checks and cost more conditions.
- REVIEW is used for invalid data so that a missing average is never read as a failure; a person must decide.
- Provision conflicts (two provisions competing for the same budget) are outside this policy and would need a ranking rule.

QUESTION 3 - Menu-Driven Order Analytics Utility (7 Marks)

3.1 Design

The utility analyses a vector of order amounts. Refunds are negative amounts, so **total revenue is net** (sales minus refunds). All missing-value and type handling sits in one helper, clean_amounts(), so no operation repeats it. Each operation is its own function, and a repeat loop with switch() selects them.

Option	Operation	Rule for unusual input
1	Total revenue (net)	empty or all NA gives 0
2 / 3	Mean / median order value	empty gives NA
4	Range (min, max)	empty gives NA, NA
5	Standard deviation	fewer than 2 values gives NA (undefined)
6	Count of refunds (amounts below 0)	NA values are removed first
7	Count above a threshold	threshold supplied by the caller
8	Number of missing values	counts NA only
9 / 10	Added later: total refund value / 10% trimmed mean	same helper, same rules
0	Exit	also ends when scripted input runs out

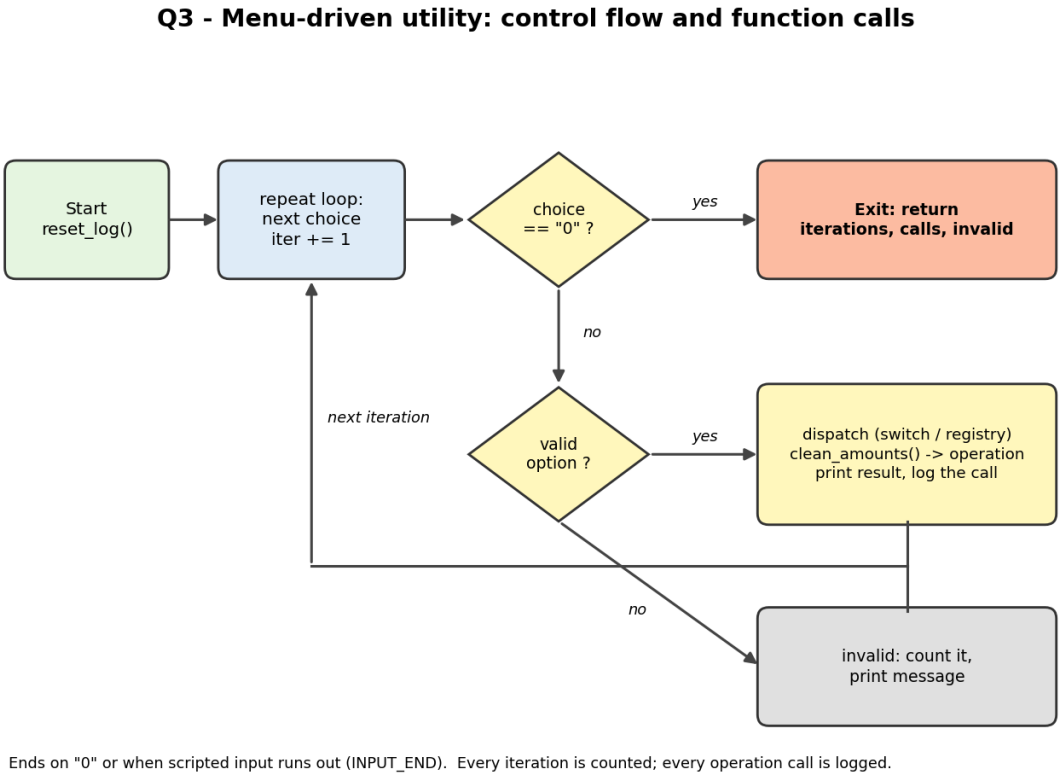


Figure 5: Menu control flow: repeat, read a choice, validate, dispatch, log

3.2 Predictions

Results for seven edge-case datasets (repeated identical amounts, refunds, an extreme outlier, missing values, one observation, empty input, all NA) and the path of four menu sessions were calculated by hand before running. Example: for the refunds dataset (100, -20, 50, -5, 30) the net total is 155, the mean 31, the median 30, the standard deviation sqrt(9020/4) = 47.4868, there are 2 refunds and 2 values above the threshold of 40.

3.3 Implementation

R code

```
# Q3: Menu-driven online order analytics utility
options(width = 85)

log_env <- new.env()
reset_log <- function() { log_env$calls <- character(); log_env$helper <- 0 }
reset_log()
log_call <- function(name) log_env$calls <- c(log_env$calls, name)

# ---- Shared helper: ALL missing-value / type handling lives here (no duplication) ----
clean_amounts <- function(x, na.rm = TRUE) {
  log_env$helper <- log_env$helper + 1
  if (!is.numeric(x)) stop("amounts must be numeric")
  if (na.rm) x <- x[!is.na(x)]
  x
}

# ---- Modular operations (net revenue: refunds are negative amounts and reduce the total) ----
total_revenue <- function(x, na.rm = TRUE) { log_call("total_revenue"); sum(clean_amounts(x, na.rm)) }
mean_order   <- function(x, na.rm = TRUE) { log_call("mean_order"); v <- clean_amounts(x, na.rm); if (length(v) == 0) NA_real_ else mean(v) }
median_order  <- function(x, na.rm = TRUE) { log_call("median_order"); v <- clean_amounts(x, na.rm); if (length(v) == 0) NA_real_ else median(v) }
order_range  <- function(x, na.rm = TRUE) { log_call("order_range"); v <- clean_amounts(x, na.rm); if (length(v) == 0) c(NA_real_, NA_real_) else range(v) }
sd_order      <- function(x, na.rm = TRUE) { log_call("sd_order"); v <- clean_amounts(x, na.rm); if (length(v) < 2) NA_real_ else sd(v) }
count_refunds <- function(x, na.rm = TRUE) { log_call("count_refunds"); sum(clean_amounts(x, na.rm) < 0) }
```

```
count_above   <- function(x, thr, na.rm = TRUE) { log_call("count_above"); sum(clean_amounts(x, na.rm) > thr)
}
count_missing <- function(x) { log_call("count_missing"); sum(is.na(x)) }

# ---- Menu, version 1: switch() selection ----
menu_labels_v1 <- c("1" = "Total revenue (net)", "2" = "Mean order value", "3" = "Median order value",
                    "4" = "Range", "5" = "Standard deviation", "6" = "Count of refunds",
                    "7" = "Count above threshold", "8" = "Number of missing values", "0" = "Exit")
dispatch_v1 <- function(choice, x, thr)
  switch(choice, "1" = total_revenue(x), "2" = mean_order(x), "3" = median_order(x), "4" = order_range(x),
         "5" = sd_order(x), "6" = count_refunds(x), "7" = count_above(x, thr), "8" = count_missing(x), NULL)

run_menu <- function(x, choices = NULL, thr = 40, dispatch = dispatch_v1, labels = menu_labels_v1, verbose =
TRUE) {
  reset_log(); iter <- 0; invalid <- 0; exit_reason <- "INPUT_END"; outputs <- list()
  repeat {                                          # loop for repeated menu requests
    if (is.null(choices)) { ch <- readline("Choose an option: ") }
    else { if (iter >= length(choices)) break; ch <- as.character(choices[iter + 1]) }
    iter <- iter + 1
    if (is.na(ch) || !nzchar(ch)) { invalid <- invalid + 1; if (verbose) cat("[", iter, "] invalid (empty)\n"); next }
    if (ch == "0") { exit_reason <- "USER_EXIT"; if (verbose) cat("[", iter, "] exit\n"); break }
    res <- dispatch(ch, x, thr)
    if (is.null(res)) { invalid <- invalid + 1; if (verbose) cat("[", iter, "] invalid option:", ch, "\n"); next }
    outputs[[length(outputs) + 1]] <- res
    if (verbose) cat("[", iter, "]", labels[ch], "->", paste(round(res, 4), collapse = " "), "\n")
  }
  list(iterations = iter, invalid = invalid, calls = log_env$calls, helper_calls = log_env$helper,
       exit = exit_reason, outputs = outputs)
}

# ---- Edge-case datasets with PREDICTED results (hand-calculated before running; threshold = 40) ----
ds <- list(
  repeated  = c(50, 50, 50, 50),
  refunds   = c(100, -20, 50, -5, 30),
  outliers  = c(20, 25, 22, 24, 5000),
  missing   = c(10, NA, 30, NA, 50),
  one_obs   = c(75),
```

```
  empty     = numeric(0),
  all_na    = c(NA_real_, NA_real_))
pred <- list(                    # total, mean, median, min, max, sd, refunds, above 40, missing
  repeated = c(200, 50, 50, 50, 50, 0, 0, 4, 0),
  refunds  = c(155, 31, 30, -20, 100, 47.4868, 2, 2, 0),
  outliers = c(5091, 1018.2, 24, 20, 5000, 2225.9, 0, 1, 0),
  missing  = c(90, 30, 30, 10, 50, 20, 0, 1, 2),
  one_obs  = c(75, 75, 75, 75, 75, NA, 0, 1, 0),
  empty    = c(0, NA, NA, NA, NA, NA, 0, 0, 0),
  all_na   = c(0, NA, NA, NA, NA, NA, 0, 0, 2))
obs_row <- function(x) { r <- order_range(x); c(total_revenue(x), mean_order(x), median_order(x), r[1], r[2],
                    sd_order(x), count_refunds(x), count_above(x, 40), count_missing(x)) }
op_names <- c("total", "mean", "median", "min", "max", "sd", "refunds", ">40", "missing")
cat("=== Predicted vs observed results for 7 edge-case datasets (9 values each) ===\n")
tab <- do.call(rbind, lapply(names(ds), function(nm) {
  o <- obs_row(ds[[nm]]); p <- pred[[nm]]
  same <- (is.na(o) & is.na(p)) | (!is.na(o) & !is.na(p) & abs(o - p) <= 0.05)
  data.frame(dataset = nm, matched = sum(same), of = 9, mismatches = if (all(same)) "-" else
paste(op_names[!same], collapse = ","))
}))
print(tab, row.names = FALSE)
cat("Total matched:", sum(tab$matched), "of", sum(tab$of), "\n")
cat("\nDetail for the refunds dataset (observed): "); print(round(setNames(obs_row(ds$refunds), op_names), 4))
cat("Detail for the outliers dataset (observed): "); print(round(setNames(obs_row(ds$outliers), op_names), 2))
cat("Observed vs predicted for empty / one-observation / all-NA inputs follow the documented rules:\n")
print(rbind(empty = setNames(obs_row(ds$empty), op_names), one_obs = setNames(obs_row(ds$one_obs),
op_names),
          all_na = setNames(obs_row(ds$all_na), op_names)))
bad <- tryCatch(total_revenue(c("a", "b")), error = function(e) conditionMessage(e))
cat("\nNon-numeric input -> error:", bad, "\n")

# ---- Menu sessions: predicted path vs observed ----
cat("\n=== Menu sessions (predicted before running) ===\n")
sessA <- c("1", "3", "x", "5", "", "0")
cat("Session A:", paste(shQuote(sessA), collapse = " "), "\n")
rA <- run_menu(ds$refunds, sessA)
cat("Predicted: iterations 6, calls 3 (total_revenue, median_order, sd_order), invalid 2, exit USER_EXIT\n")
```

```
cat("Observed : iterations", rA$iterations, ", calls", length(rA$calls), "(", paste(rA$calls, collapse = ", "), "), invalid", rA$invalid, ", exit", rA$exit, "\n")
sessB <- c("9", "10", "0")
cat("\nSession B before extension:", paste(shQuote(sessB), collapse = " "), "\n")
rB <- run_menu(ds$refunds, sessB)
cat("Predicted: iterations 3, calls 0, invalid 2\n")
cat("Observed : iterations", rB$iterations, ", calls", length(rB$calls), ", invalid", rB$invalid, "\n")
sessC <- c("2", "6")
cat("\nSession C (no exit option typed):\n")
rC <- run_menu(ds$missing, sessC)
cat("Predicted: iterations 2, calls 2, exit INPUT_END\n")
cat("Observed : iterations", rC$iterations, ", calls", length(rC$calls), ", exit", rC$exit, "\n")
sessD <- c("-1", "2.5", "abc", "08", " ")
rD <- run_menu(ds$repeated, sessD, verbose = FALSE)
cat("\nSession D invalid-only", paste(shQuote(sessD), collapse = " "), ": predicted invalid 5, calls 0; observed invalid", rD$invalid, ", calls", length(rD$calls), "\n")

# ---- Extension: add two operations and evaluate duplication ----
cat("\n=== Extension: two new operations (9 refund value, 10 trimmed mean) ===\n")
refund_value <- function(x, na.rm = TRUE) { log_call("refund_value"); v <- clean_amounts(x, na.rm); sum(v[v < 0]) }
trimmed_mean <- function(x, na.rm = TRUE, trim = 0.1) { log_call("trimmed_mean"); v <- clean_amounts(x, na.rm); if (length(v) == 0) NA_real_ else mean(v, trim = trim) }
before_lines <- length(deparse(dispatch_v1)) + length(menu_labels_v1)
dispatch_v1b <- function(choice, x, thr)
  switch(choice, "1" = total_revenue(x), "2" = mean_order(x), "3" = median_order(x), "4" = order_range(x),
         "5" = sd_order(x), "6" = count_refunds(x), "7" = count_above(x, thr), "8" = count_missing(x),
         "9" = refund_value(x), "10" = trimmed_mean(x), NULL)
labels_v1b <- c(menu_labels_v1, "9" = "Total refund value", "10" = "Trimmed mean (10%)")
cat("Places edited per new operation in the switch design: 3 (function, switch case, menu label)\n")
rB2 <- run_menu(ds$refunds, sessB, dispatch = dispatch_v1b, labels = labels_v1b)
cat("Session B after extension -> iterations", rB2$iterations, ", calls", length(rB2$calls), ", invalid", rB2$invalid, "\n")

# ---- Maintainability v2: registry-driven menu (the loop and dispatcher never change) ----
registry <- list(
  "1" = list(label = "Total revenue (net)", fun = function(x, thr) total_revenue(x)),
```

```
  "2" = list(label = "Mean order value",    fun = function(x, thr) mean_order(x)),
  "3" = list(label = "Median order value",  fun = function(x, thr) median_order(x)),
  "4" = list(label = "Range",               fun = function(x, thr) order_range(x)),
  "5" = list(label = "Standard deviation",  fun = function(x, thr) sd_order(x)),
  "6" = list(label = "Count of refunds",    fun = function(x, thr) count_refunds(x)),
  "7" = list(label = "Count above threshold", fun = function(x, thr) count_above(x, thr)),
  "8" = list(label = "Number of missing values", fun = function(x, thr) count_missing(x)),
  "9" = list(label = "Total refund value",  fun = function(x, thr) refund_value(x)),
  "10" = list(label = "Trimmed mean (10%)", fun = function(x, thr) trimmed_mean(x)))
dispatch_reg <- function(choice, x, thr) { e <- registry[[choice]]; if (is.null(e)) NULL else e$fun(x, thr) }
labels_reg <- c(setNames(sapply(registry, function(e) e$label), names(registry)), "0" = "Exit")
same <- sapply(names(ds), function(nm) {
  a <- run_menu(ds[[nm]], as.character(1:10), dispatch = dispatch_v1b, labels = labels_v1b, verbose = FALSE)$outputs
  b <- run_menu(ds[[nm]], as.character(1:10), dispatch = dispatch_reg, labels = labels_reg, verbose = FALSE)$outputs
  isTRUE(all.equal(a, b)) })
cat("Registry menu gives identical outputs for all 10 operations on all 7 datasets:", all(same), "\n")
registry[["11"]] <- list(label = "Largest order", fun = function(x, thr) { log_call("max_order"); max(clean_amounts(x)) })
r11 <- run_menu(ds$refunds, c("11", "0"), dispatch = dispatch_reg, labels = c(labels_reg, "11" = "Largest order"))
cat("Adding operation 11 to the registry: 1 registry entry, 0 changes to run_menu() or dispatch_reg(); result:", r11$outputs[[1]], "\n")

ops <- list(total_revenue, mean_order, median_order, order_range, sd_order, count_refunds, count_above, count_missing, refund_value, trimmed_mean)
nm <- c("total_revenue", "mean_order", "median_order", "order_range", "sd_order", "count_refunds", "count_above", "count_missing", "refund_value", "trimmed_mean")
src <- sapply(ops, function(f) paste(deparse(f), collapse = " "))
dup <- data.frame(operation = nm, uses_clean_amounts = grepl("clean_amounts\\(", src), own_is_na_handling = grepl("is\\.na", src),
                  own_empty_check = grepl("length\\(v\\)", src))
print(dup, row.names = FALSE)
cat("Operations with their own NA-handling code:", sum(dup$own_is_na_handling), "(count_missing needs is.na by definition)\n")

# ---- Session with call and iteration log ----
```

```
set.seed(5)
long <- c(sample(as.character(1:8), 25, TRUE), "x", "99", "0")
rL <- run_menu(ds$refunds, long, verbose = FALSE)
cat("\nLong scripted session: iterations =", rL$iterations, "| operation calls =", length(rL$calls),
    "| invalid =", rL$invalid, "| helper (clean_amounts) calls =", rL$helper_calls, "| exit =", rL$exit, "\n")
cc <- table(factor(rL$calls, levels = c("total_revenue", "mean_order", "median_order", "order_range", "sd_order",
"count_refunds", "count_above", "count_missing")))
print(cc)
png("fig/fig_q3_calls.png", width = 1700, height = 850, res = 170)
par(mar = c(8, 5, 4, 1))
barplot(cc, las = 2, col = "#6baed6", ylab = "Function calls", main = "Function calls in a 28-iteration scripted
session", cex.names = 0.8)
dev.off()
```

Output (captured from running the code above)

```
=== Predicted vs observed results for 7 edge-case datasets (9 values each) ===
  dataset matched of mismatches
 repeated       9  9         -
  refunds       9  9         -
 outliers       9  9         -
  missing       9  9         -
  one_obs       9  9         -
    empty       9  9         -
   all_na       9  9         -
Total matched: 63 of 63

Detail for the refunds dataset (observed):    total     mean   median      min      max       sd  refunds     >40  missing
155.0000  31.0000  30.0000 -20.0000 100.0000  47.4868   2.0000   2.0000   0.0000
Detail for the outliers dataset (observed):   total    mean  median     min     max      sd refunds     >40 missing
5091.00 1018.20   24.00   20.00 5000.00 2225.89    0.00    1.00    0.00
Observed vs predicted for empty / one-observation / all-NA inputs follow the documented rules:
       total mean median min max sd refunds >40 missing
empty      0   NA     NA  NA  NA NA       0   0       0
one_obs   75   75     75  75  75 NA       0   1       0
all_na     0   NA     NA  NA  NA NA       0   0       2
```

Non-numeric input -> error: amounts must be numeric

=== Menu sessions (predicted before running) ===

Session A: '1' '3' 'x' '5' '' '0'

[1] Total revenue (net) -> 155

[2] Median order value -> 30

[3] invalid option: x

[4] Standard deviation -> 47.4868

[5] invalid (empty)

[6] exit

Predicted: iterations 6, calls 3 (total_revenue, median_order, sd_order), invalid 2, exit USER_EXIT

Observed : iterations 6 , calls 3 (total_revenue, median_order, sd_order), invalid 2 , exit USER_EXIT

Session B before extension: '9' '10' '0'

[1] invalid option: 9

[2] invalid option: 10

[3] exit

Predicted: iterations 3, calls 0, invalid 2

Observed : iterations 3 , calls 0 , invalid 2

Session C (no exit option typed):

[1] Mean order value -> 30

[2] Count of refunds -> 0

Predicted: iterations 2, calls 2, exit INPUT_END

Observed : iterations 2 , calls 2 , exit INPUT_END

Session D invalid-only '-1' '2.5' 'abc' '08' ' ' : predicted invalid 5, calls 0; observed invalid 5 , calls 0

=== Extension: two new operations (9 refund value, 10 trimmed mean) ===

Places edited per new operation in the switch design: 3 (function, switch case, menu label)

[1] Total refund value -> -25

[2] Trimmed mean (10%) -> 31

[3] exit

Session B after extension -> iterations 3 , calls 2 , invalid 0

Registry menu gives identical outputs for all 10 operations on all 7 datasets: TRUE

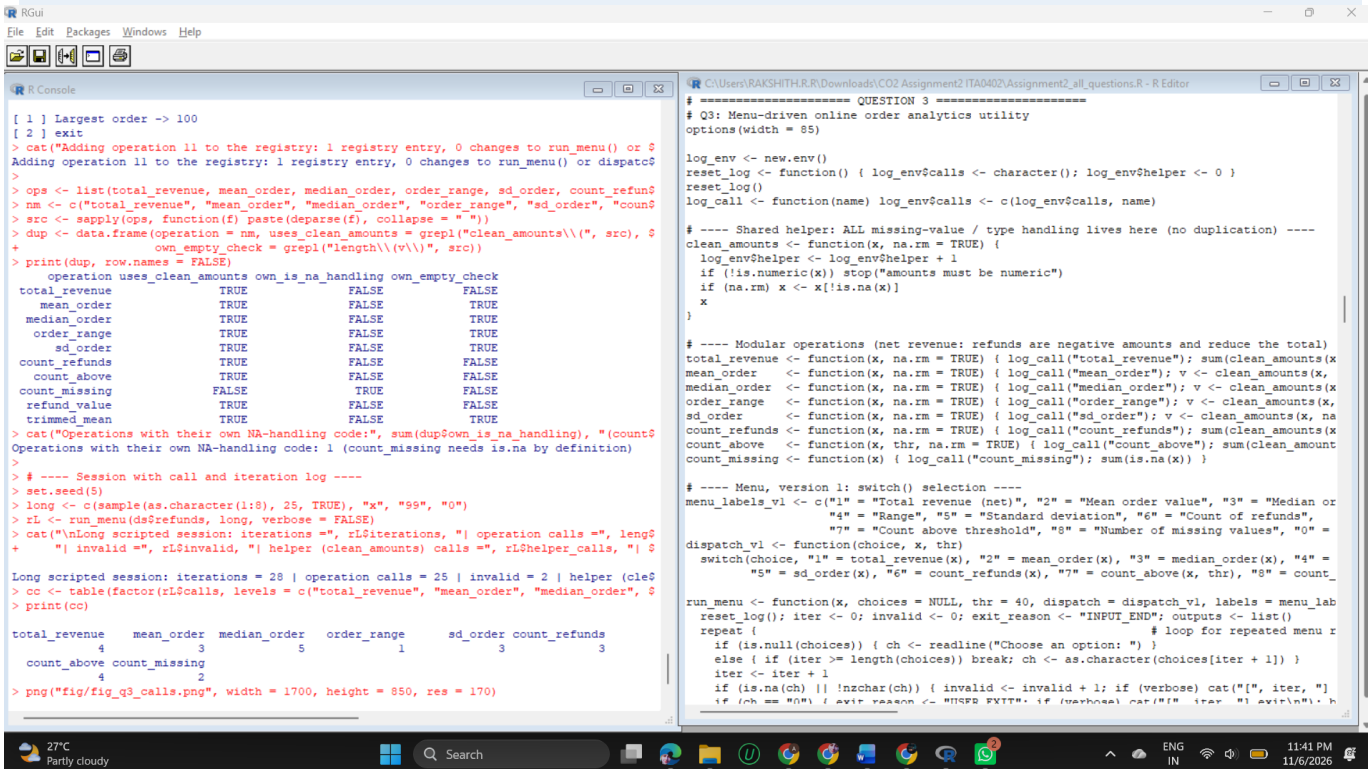
[1] Largest order -> 100

[2] exit

```
Adding operation 11 to the registry: 1 registry entry, 0 changes to run_menu() or dispatch_reg(); result: 100
     operation uses_clean_amounts own_is_na_handling own_empty_check
 total_revenue            TRUE              FALSE             FALSE
    mean_order            TRUE              FALSE             TRUE
  median_order            TRUE              FALSE             TRUE
   order_range            TRUE              FALSE             TRUE
     sd_order             TRUE              FALSE             TRUE
 count_refunds            TRUE              FALSE             FALSE
  count_above             TRUE              FALSE             FALSE
 count_missing            FALSE             TRUE              FALSE
 refund_value             TRUE              FALSE             FALSE
 trimmed_mean             TRUE              FALSE             TRUE
Operations with their own NA-handling code: 1 (count_missing needs is.na by definition)

Long scripted session: iterations = 28 | operation calls = 25 | invalid = 2 | helper (clean_amounts) calls = 23 | exit = USER_EXIT

total_revenue    mean_order  median_order   order_range      sd_order count_refunds
            4             3             5             1             3             3
 count_above count_missing
            4             2
```



```
```
```
```

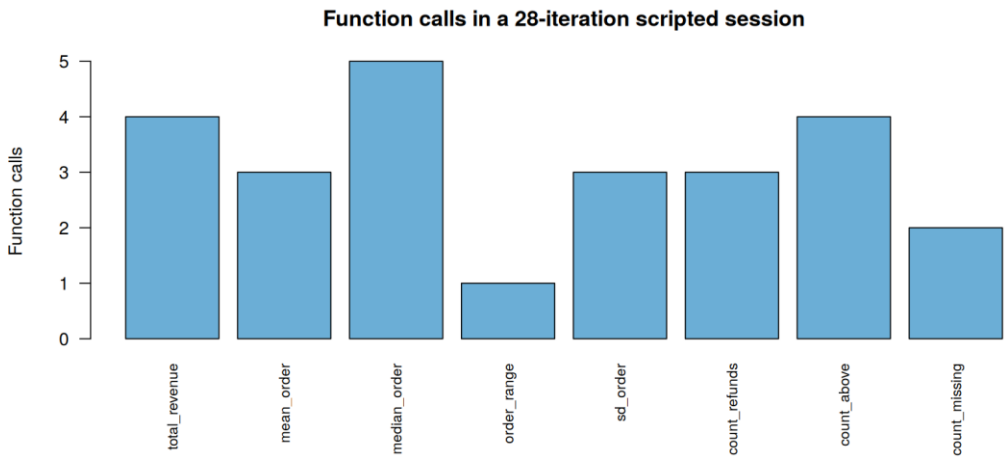


Figure 6: Function calls recorded in a scripted session of 28 menu iterations

3.4 Analysis

- **Prediction versus observation:** 63 of 63 statistics matched (7 datasets x 9 values) and all four menu sessions matched. For the outlier dataset the mean (1,018.2) is far above the median (24), and the standard deviation (2,225.9) shows the outlier's influence.
- **Session counts:** session A (1 3 x 5 (blank) 0) made 6 iterations, 3 function calls and 2 invalid entries, and ended by user exit; session C ended by input exhaustion after 2 calls; session D (-1, 2.5, abc, 08, blank) was 5 invalid entries and 0 calls. A longer session of 28 iterations produced 25 operation calls, 2 invalid entries and 23 calls of the shared helper (the 2 count_missing calls do not use it).
- **Invalid input:** an unknown option, empty text or a non-numeric string never stops the loop; it is counted and the menu continues. Non-numeric amounts stop with a clear error (amounts must be numeric).
- **Edge cases are documented, not accidental:** an empty vector gives total 0 but mean NA, because an average of nothing is undefined; one observation gives standard deviation NA for the same reason.

3.5 Extension and duplication evaluation

Design	Places to edit for one new operation	Evidence
switch() version (v1)	3: the function, a case in switch(), a label in the menu text	Two new operations (9 and 10) needed 6 edits; before the edit, options 9 and 10 were invalid (2 invalid, 0 calls); after it they ran (0 invalid, 2 calls)
Registry version (v2)	1: one entry holding the label and the function	Identical outputs for all 10 operations on all 7 datasets; an 11th operation (largest order) was added as 1 entry with no change to run_menu() or dispatch_reg()

- **Duplication check:** 9 of 10 operations use clean_amounts() and none repeats the NA-removal code; only count_missing has its own is.na call, which is its definition.
- **Remaining duplication:** five functions (mean, median, range, standard deviation, trimmed mean) each repeat the 'empty or too few values' rule. A further step would move that rule into a second helper.

QUESTION 4 - Iterative Package-Length Validation (6 Marks)

4.1 Design

A reading is valid when it lies in 1.0 to 120.0 cm inclusive. Sensors may retransmit, so up to **3 attempts** are allowed. The routine must distinguish a package that was **corrected** (an invalid reading followed by a valid one) from one that was **rejected** (no valid reading within the limit or no data left).

Disposition	Meaning
ACCEPTED	first reading valid; later retransmissions are ignored as duplicates
ACCEPTED_AFTER_CORRECTION	a valid reading arrived after one or two invalid ones
REJECTED_RETRY_LIMIT	three invalid readings used up the attempts, even if a valid reading follows
REJECTED_NO_DATA	the readings ran out before three attempts (including an empty sequence)

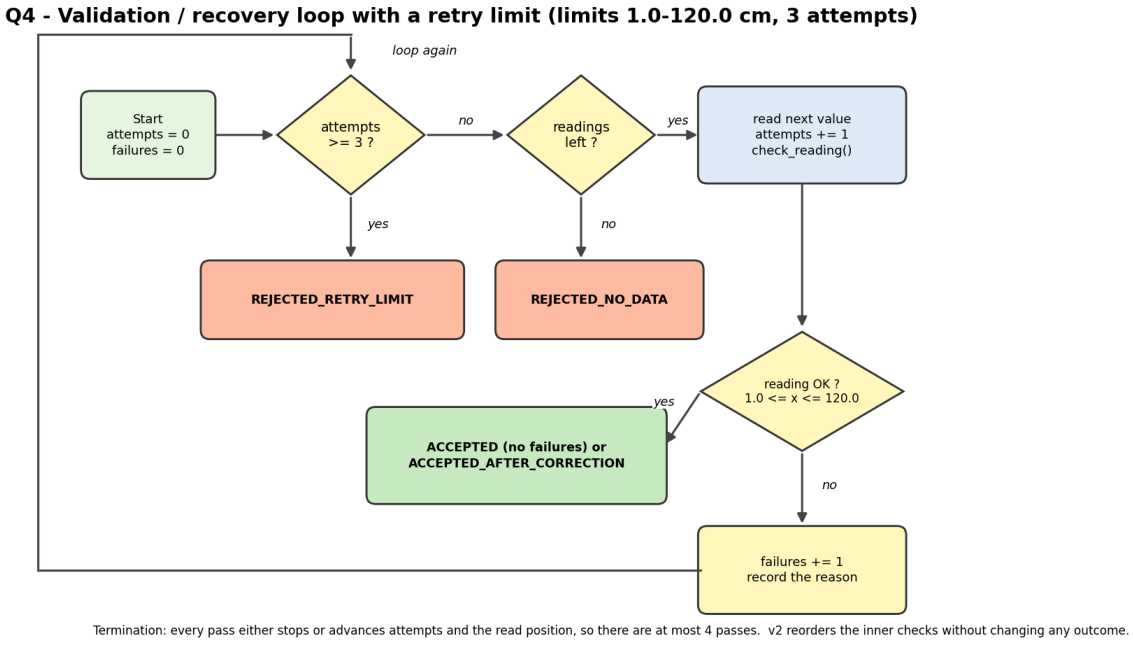


Figure 7: The `while` loop with its retry limit and the four dispositions

Termination. Every pass either sets a final status or advances the read position and the attempt counter, so the loop runs at most 4 passes (3 attempts plus one pass that detects the limit or the end of data).

Two check orders. Version 1 tests missing, blank, numeric, lower limit, upper limit. Version 2 parses once, tests the valid case first (!is.na, >= 1.0, <= 120.0) and classifies the failure reason only when needed. Both must give identical outcomes; they may differ in condition evaluations.

4.2 Test sequences and predictions

Eleven sequences were predicted (status, attempts, failures) before running: valid first reading; invalid then valid; both endpoints 1.0 and 120.0; 0.99 and 120.01 followed by 120.0 (accepted on the last allowed attempt); an alternating stream; repeated missing or non-numeric readings with a valid fourth reading; missing only; empty; duplicates; and a numeric failure at the limit.

4.3 Implementation

R code

```
# Q4: Iterative package-length validation routine
options(width = 85)
set.seed(44)
LO <- 1.0; HI <- 120.0; MAX_ATTEMPTS <- 3      # declared limits and retry policy
# Final dispositions: ACCEPTED, ACCEPTED_AFTER_CORRECTION, REJECTED_RETRY_LIMIT, REJECTED_NO_DATA

ev <- new.env(); ev$n <- 0
ck <- function(cond) { ev$n <- ev$n + 1; cond }          # counts each condition evaluated

# Version 1: missing -> blank -> numeric -> lower limit -> upper limit
check_v1 <- function(raw, lo = LO, hi = HI) {
  if (ck(is.na(raw))) return("MISSING")
  if (ck(!nzchar(trimws(as.character(raw))))) return("MISSING")
  num <- suppressWarnings(as.numeric(raw))
  if (ck(is.na(num))) return("NON_NUMERIC")
  if (ck(num < lo)) return("BELOW_MIN")
  if (ck(num > hi)) return("ABOVE_MAX")
  "OK"
}
# Version 2: parse once, test the common (valid) case first, classify the reason only on failure
check_v2 <- function(raw, lo = LO, hi = HI) {
  num <- suppressWarnings(as.numeric(raw))
  if (ck(!is.na(num)) && ck(num >= lo) && ck(num <= hi)) return("OK")
  if (ck(!is.na(num))) return(if (ck(num < lo)) "BELOW_MIN" else "ABOVE_MAX")
  if (ck(is.na(raw)) || ck(!nzchar(trimws(as.character(raw))))) "MISSING" else "NON_NUMERIC"
}
```

```
validate_sequence <- function(readings, checker, max_attempts = MAX_ATTEMPTS) {
  stopifnot(max_attempts >= 1, LO < HI)
  ev$n <- 0
  attempts <- 0; failures <- 0; i <- 1; passes <- 0; status <- NA_character_; value <- NA_real_
  reasons <- character()
  while (is.na(status)) {                          # terminates: i and attempts increase every pass
    passes <- passes + 1
    if (attempts >= max_attempts) { status <- "REJECTED_RETRY_LIMIT"; next }
    if (i > length(readings))     { status <- "REJECTED_NO_DATA"; next }
    raw <- readings[[i]]; i <- i + 1; attempts <- attempts + 1
    r <- checker(raw)
    if (r == "OK") {
      value <- as.numeric(raw)
      status <- if (failures == 0) "ACCEPTED" else "ACCEPTED_AFTER_CORRECTION"
    } else { failures <- failures + 1; reasons <- c(reasons, r) }
  }
  list(status = status, attempts = attempts, failures = failures, passes = passes,
       unread = length(readings) - (i - 1), evals = ev$n, value = value, reasons = reasons)
}

# ---- Eleven test sequences with PREDICTIONS (status, attempts, failures) written before running ----
seqs <- list(
  S1_valid_first   = list("45.5"),
  S2_invalid_valid = list(150, 80),
  S3a_low_endpoint = list(1.0),
  S3b_high_endpoint = list(120.0),
  S3c_edges_then_ok = list(0.99, 120.01, 120.0),
  S4_alternating   = list("x", 60, "y", 70),
  S5a_repeat_bad   = list(NA, "", "abc", "5"),
  S5b_missing_only = list(NA, NA),
  S6_empty         = list(),
  S7_duplicates    = list(42, 42, 42),
  S8_limit_numeric = list(0, 130, "abc", 50))
pred <- data.frame(seq = names(seqs),
  status = c("ACCEPTED", "ACCEPTED_AFTER_CORRECTION", "ACCEPTED", "ACCEPTED",
"ACCEPTED_AFTER_CORRECTION",
```

```
          "ACCEPTED_AFTER_CORRECTION",  "REJECTED_RETRY_LIMIT",  "REJECTED_NO_DATA",
"REJECTED_NO_DATA",
          "ACCEPTED", "REJECTED_RETRY_LIMIT"),
  attempts = c(1, 2, 1, 1, 3, 2, 3, 2, 0, 1, 3),
  failures = c(0, 1, 0, 0, 2, 1, 3, 2, 0, 0, 3), stringsAsFactors = FALSE)

obs1 <- lapply(seqs, validate_sequence, checker = check_v1)
obs2 <- lapply(seqs, validate_sequence, checker = check_v2)
cmp <- data.frame(seq = names(seqs),
  pred  =  paste(sub("ACCEPTED_AFTER_CORRECTION",  "ACC_CORR",  sub("REJECTED_",  "REJ_",
pred$status)), pred$attempts, pred$failures, sep = " "),
  obs  =  sapply(obs1,  function(o)  paste(sub("ACCEPTED_AFTER_CORRECTION",  "ACC_CORR",
sub("REJECTED_", "REJ_", o$status)), o$attempts, o$failures)),
  stringsAsFactors = FALSE, row.names = NULL)
cmp$ok <- ifelse(cmp$pred == cmp$obs, "yes", "NO")
cat("=== Prediction vs observation (status attempts failures; ACC_CORR = accepted after correction) ===\n")
print(cmp, row.names = FALSE)
cat("Matches:", sum(cmp$ok == "yes"), "of", nrow(cmp), "\n")

cat("\n=== Measured iterations and condition evaluations (v1 = original order, v2 = redesigned) ===\n")
m <- data.frame(seq = names(seqs), attempts = sapply(obs1, `[[`, "attempts"), loop_passes = sapply(obs1, `[[`,
"passes"),
           unread = sapply(obs1, `[[`, "unread"), evals_v1 = sapply(obs1, `[[`, "evals"), evals_v2 = sapply(obs2,
`[[`, "evals"), row.names = NULL)
print(m, row.names = FALSE)
cat("Totals: attempts =", sum(m$attempts), "| loop passes =", sum(m$loop_passes), "| evaluations v1 =",
sum(m$evals_v1), "| v2 =", sum(m$evals_v2), "\n")
same_out <- all(mapply(function(a, b) identical(a$status, b$status) && a$attempts == b$attempts && a$failures
== b$failures &&
                         identical(a$reasons, b$reasons), obs1, obs2))
cat("v1 and v2 give identical status, attempts, failures and failure reasons on all 11 sequences:", same_out, "\n")
cat("Failure reasons recorded (v1):\n")
for  (nm  in  names(seqs))  if  (length(obs1[[nm]]$reasons))  cat(sprintf("  %-18s  %s\n",  nm,
paste(obs1[[nm]]$reasons, collapse = ", ")))

cat("\n=== Evaluations per single reading, by reading type ===\n")
types <- list(valid = "50", NA_value = NA, blank = "", text = "abc", below = 0.5, above = 200)
```

```
per <- data.frame(reading = names(types), v1 = NA_integer_, v2 = NA_integer_, same_result = NA)
for (k in seq_along(types)) {
  ev$n <- 0; r1 <- check_v1(types[[k]]); per$v1[k] <- ev$n
  ev$n <- 0; r2 <- check_v2(types[[k]]); per$v2[k] <- ev$n
  per$same_result[k] <- r1 == r2
}
per$saving <- per$v1 - per$v2
print(per, row.names = FALSE)

cat("\n=== Simulation: 5,000 sequences per invalid-rate (up to 4 readings each) ===\n")
make_reading <- function(p) {
  if (runif(1) >= p) return(round(runif(1, LO, HI), 1))
  switch(sample(5, 1, prob = c(.25, .15, .20, .20, .20)), NA, "", "abc", round(runif(1, -20, LO - 0.1), 1),
round(runif(1, HI + 0.1, 300), 1))
}
rates <- c(0, 0.05, 0.1, 0.2, 0.4, 0.6, 0.8, 0.9, 0.95)
sim <- do.call(rbind, lapply(rates, function(p) {
  e1 <- e2 <- att <- 0; agree <- TRUE
  for (s in 1:5000) {
    rd <- replicate(4, make_reading(p), simplify = FALSE)
    a <- validate_sequence(rd, check_v1); b <- validate_sequence(rd, check_v2)
    e1 <- e1 + a$evals; e2 <- e2 + b$evals; att <- att + a$attempts
    if (a$status != b$status || a$attempts != b$attempts || a$failures != b$failures) agree <- FALSE
  }
  data.frame(invalid_rate = p, evals_per_attempt_v1 = round(e1 / att, 3), evals_per_attempt_v2 = round(e2 / att, 3),
outcomes_identical = agree)
}))
print(sim, row.names = FALSE)
cat("Rate at which v2 stops being cheaper (interpolated):",
    round(approx(sim$evals_per_attempt_v2 - sim$evals_per_attempt_v1, sim$invalid_rate, xout = 0)$y, 2), "\n")

png("fig/fig_q4_evals.png", width = 1700, height = 900, res = 170)
par(mar = c(5, 5, 4, 1))
plot(sim$invalid_rate, sim$evals_per_attempt_v1, type = "b", pch = 16, col = "#3182bd", ylim = c(2.5, 5), lwd =
2,
     xlab = "Probability that a reading is invalid", ylab = "Condition evaluations per reading",
     main = "Validation order: v1 (original) vs v2 (redesigned)")
```

```
lines(sim$invalid_rate, sim$evals_per_attempt_v2, type = "b", pch = 17, col = "#e6550d", lwd = 2)
legend("topright", legend = c("v1 missing > blank > numeric > range", "v2 parse once, valid case first"), col =
c("#3182bd", "#e6550d"), pch = c(16, 17), lwd = 2, bty = "n", cex = 0.85)
dev.off()
```

Output (captured from running the code above)

```
=== Prediction vs observation (status attempts failures; ACC_CORR = accepted after correction) ===
                seq               pred                obs  ok
    S1_valid_first        ACCEPTED 1 0        ACCEPTED 1 0 yes
   S2_invalid_valid        ACC_CORR 2 1        ACC_CORR 2 1 yes
   S3a_low_endpoint        ACCEPTED 1 0        ACCEPTED 1 0 yes
  S3b_high_endpoint        ACCEPTED 1 0        ACCEPTED 1 0 yes
  S3c_edges_then_ok        ACC_CORR 3 2        ACC_CORR 3 2 yes
     S4_alternating        ACC_CORR 2 1        ACC_CORR 2 1 yes
     S5a_repeat_bad REJ_RETRY_LIMIT 3 3 REJ_RETRY_LIMIT 3 3 yes
   S5b_missing_only     REJ_NO_DATA 2 2     REJ_NO_DATA 2 2 yes
          S6_empty     REJ_NO_DATA 0 0     REJ_NO_DATA 0 0 yes
      S7_duplicates        ACCEPTED 1 0        ACCEPTED 1 0 yes
   S8_limit_numeric REJ_RETRY_LIMIT 3 3 REJ_RETRY_LIMIT 3 3 yes
Matches: 11 of 11

=== Measured iterations and condition evaluations (v1 = original order, v2 = redesigned) ===
                seq attempts loop_passes unread evals_v1 evals_v2
    S1_valid_first        1           1      0        5        3
   S2_invalid_valid        2           2      0       10        8
   S3a_low_endpoint        1           1      0        5        3
  S3b_high_endpoint        1           1      0        5        3
  S3c_edges_then_ok        3           3      0       14       12
     S4_alternating        2           2      2        8        7
     S5a_repeat_bad        3           4      1        6       11
   S5b_missing_only        2           3      0        2        6
          S6_empty        0           1      0        0        0
      S7_duplicates        1           1      2        5        3
   S8_limit_numeric        3           4      1       12       13
Totals: attempts = 19 | loop passes = 23 | evaluations v1 = 72 | v2 = 69
v1 and v2 give identical status, attempts, failures and failure reasons on all 11 sequences: TRUE
```

Failure reasons recorded (v1):

S2_invalid_valid	ABOVE_MAX
S3c_edges_then_ok	BELOW_MIN, ABOVE_MAX
S4_alternating	NON_NUMERIC
S5a_repeat_bad	MISSING, MISSING, NON_NUMERIC
S5b_missing_only	MISSING, MISSING
S8_limit_numeric	BELOW_MIN, ABOVE_MAX, NON_NUMERIC

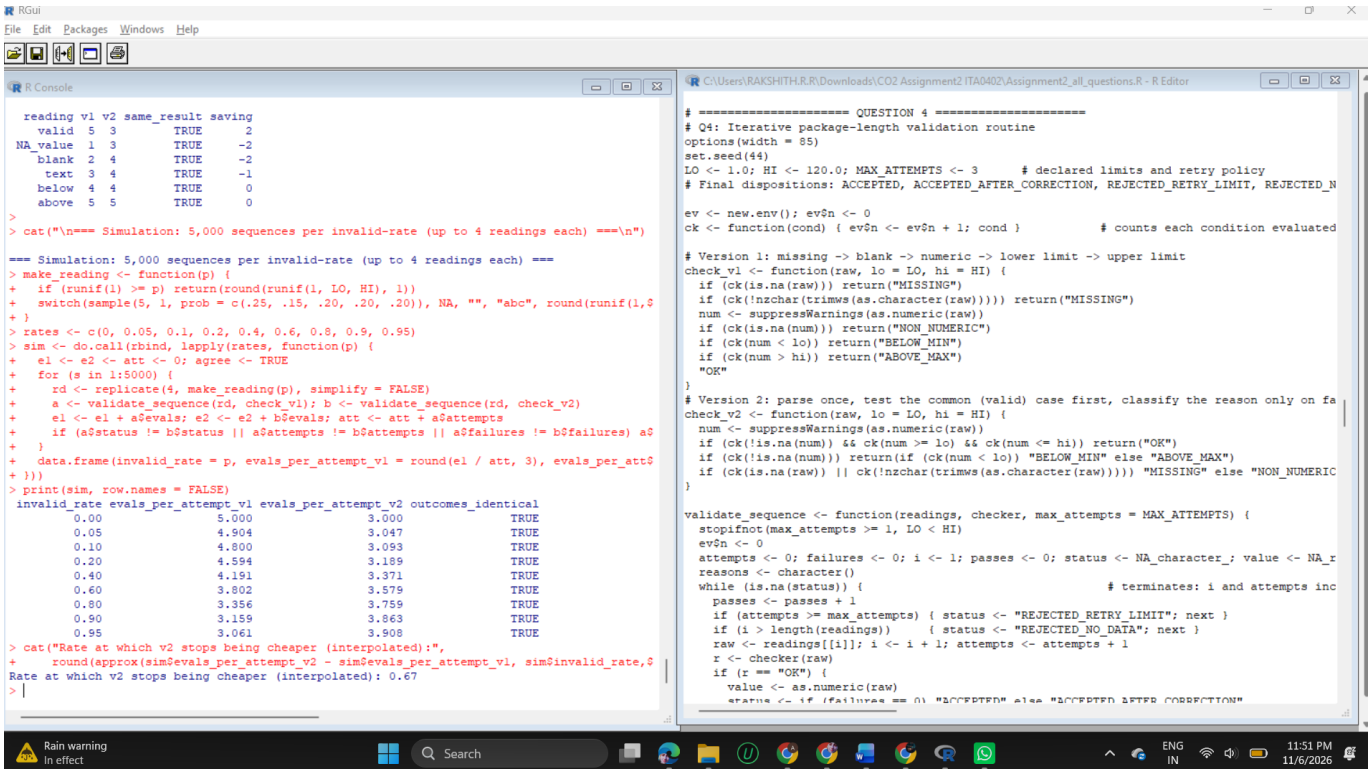
=== Evaluations per single reading, by reading type ===

reading	v1	v2	same_result	saving
valid	5	3	TRUE	2
NA_value	1	3	TRUE	-2
blank	2	4	TRUE	-2
text	3	4	TRUE	-1
below	4	4	TRUE	0
above	5	5	TRUE	0

=== Simulation: 5,000 sequences per invalid-rate (up to 4 readings each) ===

invalid_rate	evals_per_attempt_v1	evals_per_attempt_v2	outcomes_identical
0.00	5.000	3.000	TRUE
0.05	4.904	3.047	TRUE
0.10	4.800	3.093	TRUE
0.20	4.594	3.189	TRUE
0.40	4.191	3.371	TRUE
0.60	3.802	3.579	TRUE
0.80	3.356	3.759	TRUE
0.90	3.159	3.863	TRUE
0.95	3.061	3.908	TRUE

Rate at which v2 stops being cheaper (interpolated): 0.67



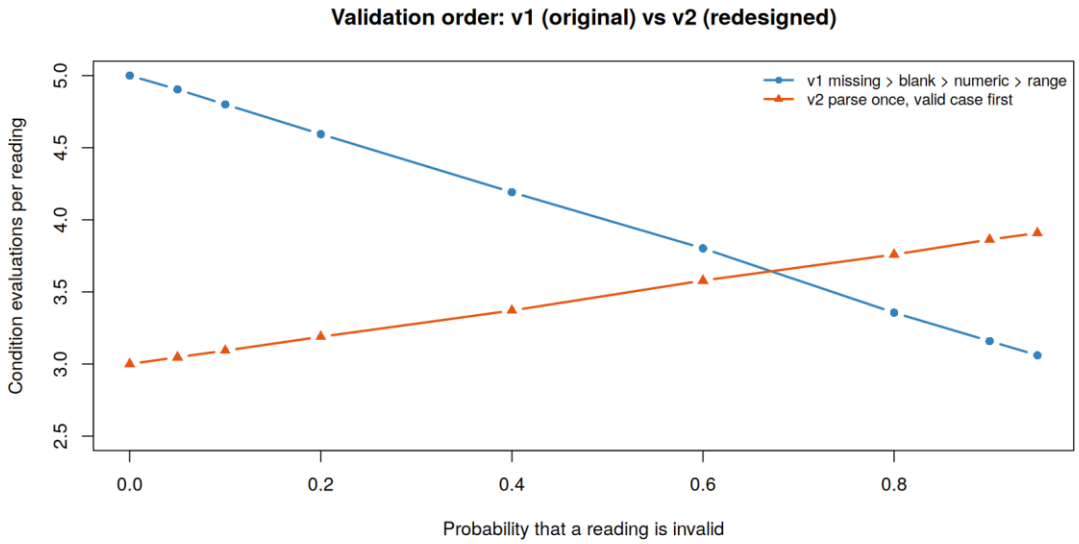


Figure 8: Mean condition evaluations per reading for the two check orders, by probability of an invalid reading

4.4 Analysis

- **Prediction versus observation:** 11 of 11 sequences matched on status, attempts and failures, including both endpoints, the correction on the third (last) allowed attempt, and the rejection in S5a where a valid fourth reading was never read because the retry limit had been reached.
- **Iterations and evaluations:** the 11 sequences needed 19 attempts and 23 loop passes (the extra passes are those that detect the limit or the end of data). Condition evaluations were 72 for v1 and 69 for v2. In S4 and S7, 2 readings each were left unread after acceptance, which is how duplicate retransmissions are ignored.

- **v1 and v2 gave identical status, attempts, failures and failure reasons** on all 11 sequences and on all 45,000 simulated sequences (9 invalid-rates x 5,000).
- **The cost depends on the data.** For a valid reading v2 needs 3 evaluations against 5 for v1 (a saving of 2). For a missing value v2 needs 3 against 1, for a blank 4 against 2 and for text 4 against 3. The simulation shows that v2 is cheaper while fewer than about 67% of readings are invalid (3.00 against 5.00 evaluations per reading at 0% invalid, 3.91 against 3.06 at 95%).
- **Honest correction of my own estimate:** by hand I expected the crossover near 83% invalid; the measured value is 67% because v2 re-tests !is.na on the failure path. The measurement replaces the estimate.

4.5 Evaluation

- Real sensor streams are mostly valid, so v2 is the better order. If a feed were mostly broken, v1 would be cheaper, which is useful to know when the data source changes.
- The strict retry limit can reject a package whose fourth reading is valid (S5a, S8). This protects against endless retries but loses a good reading; the limit is a policy value and can be raised.

QUESTION 5 - Recursive File Counter for a Nested Archive (7 Marks)

5.1 Model and recursive definition

A folder is an R list(); a file is any element that is **not** a list. Folders can be empty, can nest to any depth, and can contain NULL (a broken link, counted as nothing).

Case	Condition	Result
Base case 1	node is NULL	0
Base case 2	node is not a list (a file)	1
Base case 3	node is an empty list	0
Recursive case	node is a non-empty list	sum of count_files(child) over all children

Why it terminates. Every recursive call receives a child, which is strictly smaller than its parent folder. The nesting is finite, so every chain of calls ends at a file, a NULL or an empty folder.

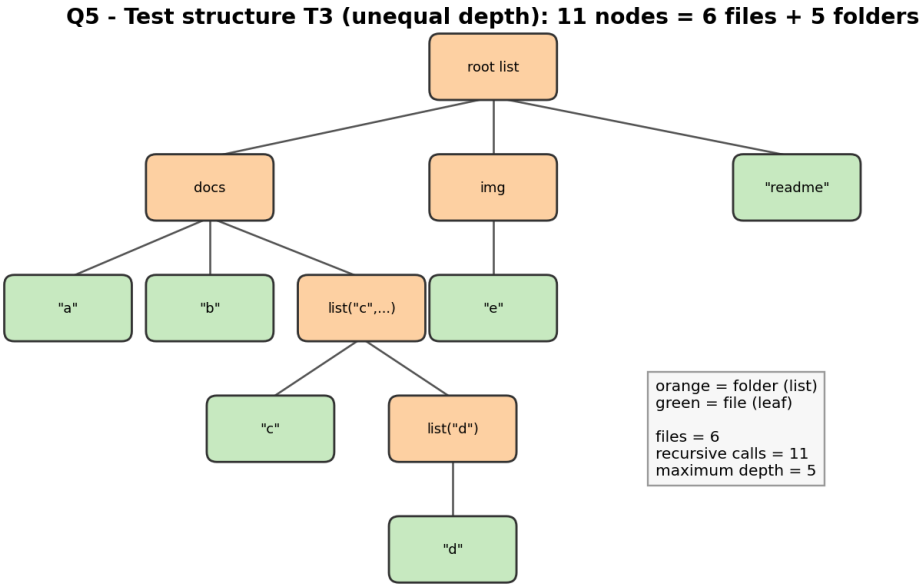


Figure 9: Test structure T3 with 6 files, 5 folders, 11 nodes and maximum depth 5

5.2 Predictions

For each of seven structures I counted by hand the files, the number of recursive calls (every node is visited exactly once, so calls equal the number of nodes) and the maximum depth. For complete trees with branching b and depth d the formulas are files = b^d and calls = (b^(d+1) - 1) / (b - 1).

5.3 Implementation

R code

```
# Q5: Recursive analysis of a nested file-and-folder archive
options(width = 85)
# MODEL: a folder is an R list(); a file is any NON-list element (here a character string).
# Folders may be empty, nested to any depth, and may contain NULL entries (broken links) that count as nothing.

calls <- new.env(); calls$n <- 0; calls$max_depth <- 0
count_files <- function(node, depth = 1) {
  calls$n <- calls$n + 1                          # every call (file, folder, NULL) is counted
  if (depth > calls$max_depth) calls$max_depth <- depth
  if (is.null(node)) return(0)                     # base case 1: broken link, nothing to count
  if (!is.list(node)) return(1)                    # base case 2: a file leaf counts as one
  if (length(node) == 0) return(0)                     # base case 3: empty folder contributes zero
  total <- 0
  for (child in node) total <- total + count_files(child, depth + 1)   # recursive case: sum over children
  total
}
count_files_naive <- function(node) {                  # first draft WITHOUT the NULL base case
  if (!is.list(node)) return(1)
  if (length(node) == 0) return(0)
  total <- 0; for (child in node) total <- total + count_files_naive(child); total
}
run_rec <- function(tree) { calls$n <- 0; calls$max_depth <- 0; f <- count_files(tree); list(files = f, calls = calls$n,
depth = calls$max_depth) }

# ---- Iterative alternative: explicit stack (pop one node at a time) ----
count_files_stack <- function(root) {
  stack <- list(root); files <- 0; pops <- 0
  while (length(stack) > 0) {
    node <- stack[[length(stack)]]; stack[[length(stack)]] <- NULL; pops <- pops + 1
    if (is.null(node)) next
    if (!is.list(node)) files <- files + 1
    else if (length(node) > 0) stack <- c(stack, node)
  }
  list(files = files, pops = pops)
```

```
}

# ---- Test structures ----
chain <- list("deep.txt"); for (i in 1:4) chain <- list(chain)       # 5 nested folders, 1 file
tests <- list(
  T1_shallow    = list("a.txt", "b.txt", "c.txt"),
  T2_deep_chain = chain,
  T3_unequal    = list(docs = list("a", "b", list("c", list("d"))), img = list("e"), "readme"),
  T4_empty      = list(empty1 = list(), a = list("x", list()), "y"),
  T5_single     = list("only.txt"),
  T6_null_link  = list("a", NULL),
  T7_all_empty  = list())
# PREDICTIONS (counted by hand before running): files, calls = number of nodes visited, depth
pred <- data.frame(test = names(tests), files = c(3, 1, 6, 2, 1, 1, 0), calls = c(4, 6, 11, 6, 2, 3, 1), depth = c(2, 6, 6,
4, 2, 2, 1))
obs <- do.call(rbind, lapply(tests, function(t) as.data.frame(run_rec(t))))
cmp <- data.frame(test = names(tests), pred_files = pred$files, obs_files = obs$files, pred_calls = pred$calls,
obs_calls = obs$calls,
                  pred_depth = pred$depth, obs_depth = obs$depth, row.names = NULL)
cmp$ok <- ifelse(cmp$pred_files == cmp$obs_files & cmp$pred_calls == cmp$obs_calls & cmp$pred_depth ==
cmp$obs_depth, "yes", "NO")
cat("=== Recursive count: predicted vs observed (files, recursive calls, maximum depth) ===\n")
print(cmp, row.names = FALSE)
cat("Matches:", sum(cmp$ok == "yes"), "of", nrow(cmp), "\n")

cat("\n=== Base-case diagnosis: draft without the NULL base case ===\n")
naive <- sapply(tests, function(t) tryCatch(count_files_naive(t), error = function(e) NA))
print(data.frame(test = names(tests), correct = obs$files, naive_draft = naive, row.names = NULL), row.names =
FALSE)

cat("\n=== Recursive vs iterative stack: same answers and same number of visits ===\n")
st <- do.call(rbind, lapply(tests, function(t) as.data.frame(count_files_stack(t))))
print(data.frame(test = names(tests), rec_files = obs$files, stack_files = st$files, rec_calls = obs$calls, stack_pops
= st$pops,
                 equal = obs$files == st$files & obs$calls == st$pops, row.names = NULL), row.names = FALSE)

# ---- Effect of depth and branching: complete trees ----
```

```
make_tree <- function(d, b) if (d == 0) "f" else lapply(seq_len(b), function(i) make_tree(d - 1, b))
grid <- data.frame(depth = c(2, 3, 3, 4, 5, 2, 6), branching = c(2, 2, 3, 3, 2, 10, 2))
grid$pred_files <- grid$branching^grid$depth
grid$pred_calls <- (grid$branching^(grid$depth + 1) - 1) / (grid$branching - 1)
res <- lapply(seq_len(nrow(grid)), function(i) run_rec(make_tree(grid$depth[i], grid$branching[i])))
grid$obs_files <- sapply(res, `[[`, "files"); grid$obs_calls <- sapply(res, `[[`, "calls"); grid$max_depth <-
sapply(res, `[[`, "depth")
grid$ok <- ifelse(grid$pred_files == grid$obs_files & grid$pred_calls == grid$obs_calls, "yes", "NO")
cat("\n=== Complete trees: formulas files = b^d and calls = (b^(d+1)-1)/(b-1), predicted before running ===\n")
print(grid, row.names = FALSE)
cat("Calls grow with the NUMBER OF NODES (branching); maximum stack depth grows with DEPTH only.\n")

# ---- Depth limit of recursion vs iteration ----
build_chain <- function(d) { x <- "leaf.txt"; for (i in seq_len(d)) x <- list(x); x }
depths <- c(100, 250, 500, 750, 1000, 1500, 5000, 20000)
dl <- do.call(rbind, lapply(depths, function(d) {
  tr <- build_chain(d)
  r <- tryCatch({ calls$n <- 0; calls$max_depth <- 0; count_files(tr); "ok" }, error = function(e)
substr(conditionMessage(e), 1, 40))
  s <- tryCatch({ count_files_stack(tr)$files; "ok" }, error = function(e) substr(conditionMessage(e), 1, 40))
  data.frame(chain_depth = d, recursive = r, stack_iterative = s)
}))
cat("\n=== Chain of nested folders: recursion versus the explicit stack ===\n")
print(dl, row.names = FALSE)
ok_depths <- depths[dl$recursive == "ok"]
cat("Deepest chain the recursive version handled in this test:", max(ok_depths), "\n")

# ---- Timing on a larger complete tree (supporting evidence) ----
big <- make_tree(8, 3)                          # 3^8 = 6,561 files, 9,841 nodes
t_rec <- median(replicate(3, system.time(count_files(big))[["elapsed"]]))
t_stk <- median(replicate(3, system.time(count_files_stack(big))[["elapsed"]]))
cat("\nComplete tree depth 8, branching 3:", run_rec(big)$files, "files,", run_rec(big)$calls, "calls\n")
cat("Elapsed (median of 3): recursive =", round(t_rec, 3), "s | stack =", round(t_stk, 3), "s\n")

png("fig/fig_q5_growth.png", width = 1800, height = 850, res = 170)
par(mfrow = c(1, 2), mar = c(5, 5, 4, 1))
```

```
plot(grid$pred_calls, grid$obs_calls, pch = 16, col = "#3182bd", cex = 1.4, log = "xy", xlab = "Predicted calls (formula)", ylab = "Observed calls",
     main = "Calls follow the node-count formula"); abline(0, 1, lty = 2)
plot(grid$depth, grid$max_depth, pch = 17, col = "#e6550d", cex = 1.4, xlab = "Tree depth d", ylab = "Maximum recursion depth observed",
     main = "Recursion depth tracks tree depth"); abline(1, 1, lty = 2)
dev.off()
```

Output (captured from running the code above)

```
=== Recursive count: predicted vs observed (files, recursive calls, maximum depth) ===
         test pred_files obs_files pred_calls obs_calls pred_depth obs_depth  ok
   T1_shallow          3         3          4         4          2         2 yes
 T2_deep_chain         1         1          6         6          6         6 yes
   T3_unequal          6         6         11        11          6         5  NO
    T4_empty           2         2          6         6          4         3  NO
    T5_single          1         1          2         2          2         2 yes
 T6_null_link          1         1          3         3          2         2 yes
 T7_all_empty          0         0          1         1          1         1 yes
Matches: 5 of 7

=== Base-case diagnosis: draft without the NULL base case ===
         test correct naive_draft
   T1_shallow       3           3
 T2_deep_chain      1           1
   T3_unequal       6           6
    T4_empty        2           2
    T5_single       1           1
 T6_null_link       1           2
 T7_all_empty       0           0

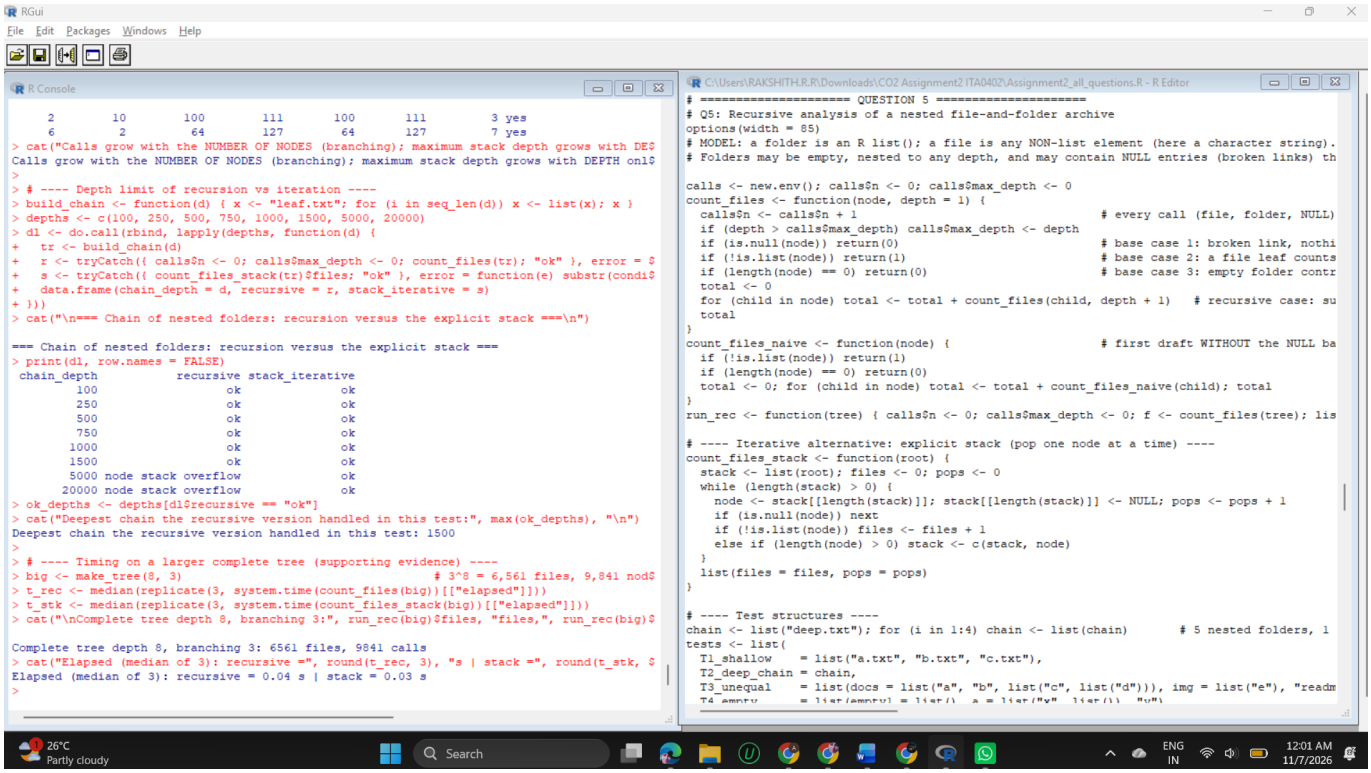
=== Recursive vs iterative stack: same answers and same number of visits ===
         test rec_files stack_files rec_calls stack_pops equal
   T1_shallow         3           3         4          4  TRUE
 T2_deep_chain        1           1         6          6  TRUE
   T3_unequal         6           6        11         11  TRUE
    T4_empty          2           2         6          6  TRUE
```

```
    T5_single        1          1        2          2  TRUE
 T6_null_link        1          1        3          3  TRUE
 T7_all_empty        0          0        1          1  TRUE

=== Complete trees: formulas files = b^d and calls = (b^(d+1)-1)/(b-1), predicted before running ===
 depth branching pred_files pred_calls obs_files obs_calls max_depth  ok
     2         2          4          7         4         7         3 yes
     3         2          8         15         8        15         4 yes
     3         3         27         40        27        40         4 yes
     4         3         81        121        81       121         5 yes
     5         2         32         63        32        63         6 yes
     2        10        100        111       100       111         3 yes
     6         2         64        127        64       127         7 yes
Calls grow with the NUMBER OF NODES (branching); maximum stack depth grows with DEPTH only.

=== Chain of nested folders: recursion versus the explicit stack ===
 chain_depth                        recursive stack_iterative
         100                             ok            ok
         250                             ok            ok
         500                             ok            ok
         750                             ok            ok
        1000 C stack usage  7971012 is too close to t              ok
        1500 C stack usage  7971012 is too close to t              ok
        5000 C stack usage  7971012 is too close to t              ok
       20000 C stack usage  7971012 is too close to t               ok
Deepest chain the recursive version handled in this test: 750

Complete tree depth 8, branching 3: 6561 files, 9841 calls
Elapsed (median of 3): recursive = 0.007 s | stack = 0.004 s
```



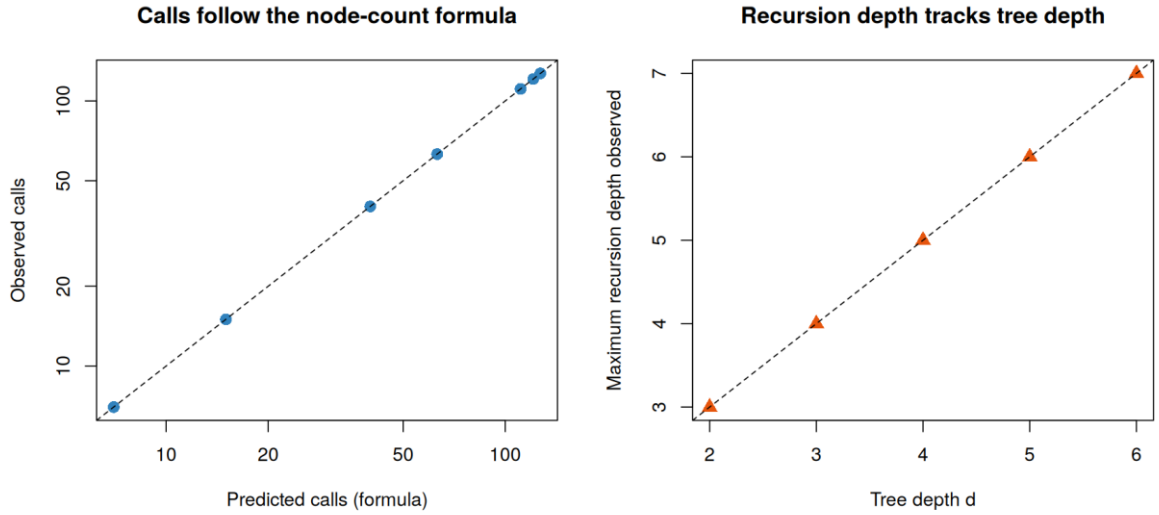


Figure 10: Left: observed calls against the node-count formula. Right: recursion depth against tree depth

5.4 Analysis

- **Prediction versus observation:** files and calls matched for all 7 structures, and all 7 complete-tree formulas matched exactly. **Two depth predictions were wrong**: for T3 I predicted 6 and the observed depth is 5; for T4 I predicted 4 and it is 3. I had counted one extra level when listing the folders; the diagram of T3 confirms depth 5.
- **Base-case diagnosis:** the first draft without the NULL base case counted a broken link as a file and returned 2 for T6 instead of 1. This is why the NULL case is explicit.

- **Effect of depth and branching:** the number of calls grows with the number of nodes, which grows with branching (b = 10, d = 2 needs 111 calls; b = 2, d = 6 needs 127). The maximum recursion depth grows only with the depth of the tree (d + 1 for a complete tree). Wide trees are therefore cheap on the stack, deep trees are not.
- **Recursion versus an explicit stack:** both gave the same file count and the same number of visits (calls = pops) on all 7 structures. On a complete tree of depth 8 and branching 3 (9,841 nodes) the times were 0.007 s and 0.004 s, so speed does not separate them.
- **Depth limit (the real difference):** a chain of nested folders was handled by recursion up to depth 750 in this test and failed from depth 1000 with C stack usage ... too close to the limit, while the stack version handled every depth tested up to 20,000. Question 7 measures this limit more carefully.

5.5 Evaluation

Recursion matches the definition of the problem and is the clearest code for ordinary archives, whose depth is small. Its weakness is that depth is limited by the call stack, not by memory. An archive built or generated automatically can be deeper than the limit, which is why the explicit-stack alternative was tested.

QUESTION 6 - Geometric Series: Iteration versus Recursion (7 Marks)

6.1 Mathematical definition and assumptions

Item	Definition
Terms	t_1 = a, t_k = r x t_(k-1), so t_k = a r^(k-1)
Partial sum	S_n = t_1 + ... + t_n, with S_0 = 0 (empty sum)
Recurrence used by the recursion	S_n = a + r x S_(n-1) for n >= 1
Closed form (used only as a check)	S_n = a (1 - r^n) / (1 - r) for r != 1; S_n = n a for r = 1
Valid inputs	n a whole number >= 0; a and r any real numbers
Base case and progress	n = 0 returns 0; each call reduces n by one, so every valid n reaches the base case

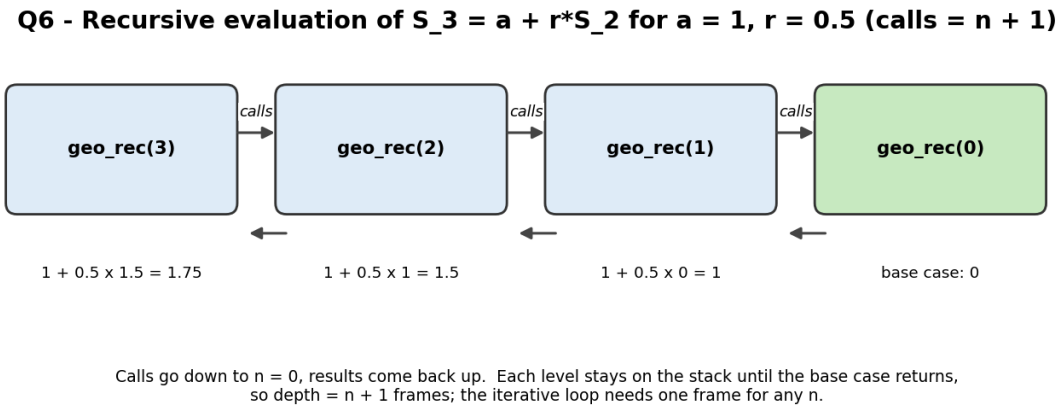


Figure 11: Recursive evaluation for n = 3: calls descend to the base case, results return upwards

Predictions. For n terms I predicted n loop iterations and n + 1 recursive calls (the base case is a call). Sums were calculated from the closed form: for r = 0.5, a = 1 the sums are 1 (n = 1), 1.9375 (n = 5) and 1.998046875 (n = 10); for r = 1, a = 3, n = 5 the sum is 15; for r = -1 the sum alternates between 1 (odd n) and 0 (even n); for r = 2 and n = 10 it is 1023.

6.2 Implementation

R code

```
# Q6: Iterative vs recursive evaluation of a geometric series
options(width = 85)
# DEFINITION. First term a, common ratio r (any real number), number of terms n (integer, n >= 0).
#   term recurrence : t_1 = a,  t_k = r * t_{k-1}              (so t_k = a * r^(k-1))
```

```
#   partial sum     : S_0 = 0 (empty sum),  S_n = a + r * S_{n-1}    for n >= 1
#   closed form     : S_n = a * (1 - r^n) / (1 - r)  for r != 1 ;  S_n = n * a  for r = 1
# Base case: n = 0 gives 0. Each recursive call reduces n by one, so it reaches the base case for every valid n.

geo_iter <- function(n, a, r) {
  if (!is.numeric(n) || length(n) != 1 || is.na(n) || n < 0 || n != floor(n)) stop("n must be a whole number >= 0")
  total <- 0; term <- a; iters <- 0
  for (k in seq_len(n)) { total <- total + term; term <- term * r; iters <- iters + 1 }
  list(sum = total, count = iters)
}
rc <- new.env(); rc$calls <- 0
geo_rec_inner <- function(n, a, r) {
  rc$calls <- rc$calls + 1
  if (n == 0) return(0)                       # base case
  a + r * geo_rec_inner(n - 1, a, r)          # recursive case
}
geo_rec <- function(n, a, r) {
  if (!is.numeric(n) || length(n) != 1 || is.na(n) || n < 0 || n != floor(n)) stop("n must be a whole number >= 0")
  rc$calls <- 0; s <- geo_rec_inner(n, a, r); list(sum = s, count = rc$calls)
}
geo_closed <- function(n, a, r) if (r == 1) n * a else a * (1 - r^n) / (1 - r)

# ---- Tests: PREDICTIONS (sum, iterations, recursive calls) written before running ----
tests <- data.frame(case = c("n=0", "n=1", "n=5", "n=10", "n=100", "n=1000 (r=0.5)", "r=1, n=5", "r=-1, n=5",
"r=-1, n=6", "r=0, n=5", "r=2, n=10", "r=2, n=20"),
                    n = c(0, 1, 5, 10, 100, 1000, 5, 5, 6, 5, 10, 20),
                    a = c(1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 1, 1),
                    r = c(.5, .5, .5, .5, .5, .5, 1, -1, -1, 0, 2, 2), stringsAsFactors = FALSE)
tests$pred_sum   <- c(0, 1, 1.9375, 1.998046875, 2, 2, 15, 1, 0, 1, 1023, 1048575)
tests$pred_iter  <- tests$n
tests$pred_calls <- tests$n + 1
obs <- lapply(seq_len(nrow(tests)), function(i) list(it = geo_iter(tests$n[i], tests$a[i], tests$r[i]), re =
tryCatch(geo_rec(tests$n[i], tests$a[i], tests$r[i]), error = function(e) NULL)))
tests$iter_sum <- sapply(obs, function(o) o$it$sum); tests$iter_count <- sapply(obs, function(o) o$it$count)
tests$rec_sum <- sapply(obs, function(o) if (is.null(o$re)) NA else o$re$sum); tests$rec_calls <- sapply(obs,
function(o) if (is.null(o$re)) NA else o$re$count)
tests$closed <- mapply(geo_closed, tests$n, tests$a, tests$r)
```

```
cat("=== Predicted vs observed sums ===\n")
print(data.frame(case = tests$case, predicted = tests$pred_sum, iterative = tests$iter_sum, recursive = tests$rec_sum, closed_form = tests$closed,
                 sums_ok = ifelse(is.na(tests$rec_sum), "RECURSION FAILED", ifelse(abs(tests$pred_sum - tests$iter_sum) < 1e-9 & abs(tests$pred_sum - tests$rec_sum) < 1e-9, "yes", "NO"))), row.names = FALSE)
cat("\n=== Predicted vs observed iterations and recursive calls ===\n")
print(data.frame(case = tests$case, pred_iter = tests$pred_iter, obs_iter = tests$iter_count, pred_calls = tests$pred_calls, obs_calls = tests$rec_calls,
                 counts_ok = ifelse(is.na(tests$rec_calls), "RECURSION FAILED", ifelse(tests$pred_iter == tests$iter_count & tests$pred_calls == tests$rec_calls, "yes", "NO"))), row.names = FALSE)
cat("\nLargest |iterative - recursive| over these tests:", format(max(abs(tests$iter_sum - tests$rec_sum), na.rm = TRUE), digits = 3), "\n")
cat("Largest |iterative - closed form|:", format(max(abs(tests$iter_sum - tests$closed)), digits = 3), "\n")

cat("\n=== Increasing term counts (a=1, r=0.5): counts grow linearly ===\n")
ns <- c(0, 1, 10, 100, 500, 750)
inc <- data.frame(n = ns, iterations = sapply(ns, function(n) geo_iter(n, 1, .5)$count), rec_calls = sapply(ns, function(n) geo_rec(n, 1, .5)$count),
                  sum_iter = sapply(ns, function(n) geo_iter(n, 1, .5)$sum), sum_rec = sapply(ns, function(n) geo_rec(n, 1, .5)$sum))
inc$abs_diff <- abs(inc$sum_iter - inc$sum_rec)
print(inc, digits = 10, row.names = FALSE)

cat("\n=== Ratio boundary cases ===\n")
cat("r = 1 : closed form (1-r^n)/(1-r) would be 0/0, so the n*a branch gives", geo_closed(5, 1, 1), "(handled by the n*a branch); iterative n=5, a=3 gives", geo_iter(5, 3, 1)$sum, "\n")
cat("r = -1: alternating partial sums 1,0,1,0 -> n=5:", geo_iter(5, 1, -1)$sum, " n=6:", geo_iter(6, 1, -1)$sum, " (no convergence, no error)\n")
cat("|r| < 1 converges to a/(1-r) = 2 for r = 0.5; S_100 =", geo_iter(100, 1, .5)$sum, "\n")
ov <- sapply(c(1023, 1024, 1025), function(n) geo_iter(n, 1, 2)$sum)
cat("r = 2 overflow boundary, S_n for n = 1023, 1024, 1025:", format(ov, digits = 4), "(n = 1024 overflows double precision to Inf)\n")
cat("Recursive version at the same boundary (n = 1024) -> ", tryCatch(format(geo_rec(1024, 1, 2)$sum), error = function(e) "ERROR: recursion limit reached before the numbers overflow"), "\n")

cat("\n=== Termination and depth ===\n")
```

```
for (bad in list(-1, 2.5, NA)) cat("geo_rec(", format(bad), ") ->", tryCatch({ geo_rec(bad, 1, .5); "accepted" }, error = function(e) conditionMessage(e)), "\n")
unguarded <- function(n) if (n == 0) 0 else 1 + unguarded(n - 1)
cat("Unguarded version with n = -1 ->", tryCatch(unguarded(-1), error = function(e) paste("never reaches the base case:", substr(conditionMessage(e), 1, 45))), "\n")
ds <- c(500, 750, 800, 900, 1000, 2000)
dd <- data.frame(n = ds, recursive = sapply(ds, function(n) tryCatch({ geo_rec(n, 1, .5); "ok" }, error = function(e) "FAILED (stack limit)")),
                 iterative = sapply(ds, function(n) tryCatch({ geo_iter(n, 1, .5); "ok" }, error = function(e) "FAILED")))
print(dd, row.names = FALSE)
cat("Iterative version at n = 1e6:", format(geo_iter(1e6, 1, .5)$sum), "with", geo_iter(1e6, 1, .5)$count, "iterations\n")

cat("\n=== Elapsed time (supporting evidence): R 4.3.3, single core, median of 5 repeats, a=1 r=0.5 ===\n")
tm <- function(f, reps) median(replicate(5, system.time(for (j in seq_len(reps)) f())[["elapsed"]])) / reps * 1e6
tt <- data.frame(n = c(10, 100, 500, 750))
tt$iterative_us <- sapply(tt$n, function(n) tm(function() geo_iter(n, 1, .5), 2000))
tt$recursive_us <- sapply(tt$n, function(n) tm(function() geo_rec(n, 1, .5), 2000))
tt$ratio <- round(tt$recursive_us / tt$iterative_us, 1)
tt$iterative_us <- round(tt$iterative_us, 1); tt$recursive_us <- round(tt$recursive_us, 1)
print(tt, row.names = FALSE)

png("fig/fig_q6_compare.png", width = 1800, height = 850, res = 170)
par(mfrow = c(1, 2), mar = c(5, 5, 4, 1))
plot(inc$n, inc$rec_calls, type = "b", pch = 17, col = "#e6550d", lwd = 2, xlab = "Number of terms n", ylab = "Count",
    main = "Work grows linearly in n")
lines(inc$n, inc$iterations, type = "b", pch = 16, col = "#3182bd", lwd = 2)
legend("topleft", legend = c("recursive calls (n+1)", "loop iterations (n)"), col = c("#e6550d", "#3182bd"), pch = c(17, 16), lwd = 2, bty = "n", cex = 0.8)
plot(tt$n, tt$recursive_us, type = "b", pch = 17, col = "#e6550d", lwd = 2, xlab = "Number of terms n", ylab = "Microseconds per evaluation",
    main = "Elapsed time (supporting evidence)", ylim = c(0, max(tt$recursive_us) * 1.1))
lines(tt$n, tt$iterative_us, type = "b", pch = 16, col = "#3182bd", lwd = 2)
legend("topleft", legend = c("recursive", "iterative"), col = c("#e6550d", "#3182bd"), pch = c(17, 16), lwd = 2, bty = "n", cex = 0.8)
dev.off()
```

Output (captured from running the code above)

```
=== Predicted vs observed sums ===
          case    predicted    iterative    recursive  closed_form         sums_ok
           n=0 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00             yes
           n=1 1.000000e+00 1.000000e+00 1.000000e+00 1.000000e+00             yes
           n=5 1.937500e+00 1.937500e+00 1.937500e+00 1.937500e+00             yes
          n=10 1.998047e+00 1.998047e+00 1.998047e+00 1.998047e+00             yes
         n=100 2.000000e+00 2.000000e+00 2.000000e+00 2.000000e+00             yes
 n=1000 (r=0.5) 2.000000e+00 2.000000e+00           NA 2.000000e+00 RECURSION FAILED
       r=1, n=5 1.500000e+01 1.500000e+01 1.500000e+01 1.500000e+01             yes
      r=-1, n=5 1.000000e+00 1.000000e+00 1.000000e+00 1.000000e+00             yes
      r=-1, n=6 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00             yes
       r=0, n=5 1.000000e+00 1.000000e+00 1.000000e+00 1.000000e+00             yes
      r=2, n=10 1.023000e+03 1.023000e+03 1.023000e+03 1.023000e+03             yes
      r=2, n=20 1.048575e+06 1.048575e+06 1.048575e+06 1.048575e+06             yes

=== Predicted vs observed iterations and recursive calls ===
          case pred_iter obs_iter pred_calls obs_calls       counts_ok
           n=0         0        0          1         1             yes
           n=1         1        1          2         2             yes
           n=5         5        5          6         6             yes
          n=10        10       10         11        11             yes
         n=100       100      100        101       101             yes
 n=1000 (r=0.5)      1000     1000       1001        NA RECURSION FAILED
       r=1, n=5         5        5          6         6             yes
      r=-1, n=5         5        5          6         6             yes
      r=-1, n=6         6        6          7         7             yes
       r=0, n=5         5        5          6         6             yes
      r=2, n=10        10       10         11        11             yes
      r=2, n=20        20       20         21        21             yes

Largest |iterative - recursive| over these tests: 0
Largest |iterative - closed form|: 0

=== Increasing term counts (a=1, r=0.5): counts grow linearly ===
```

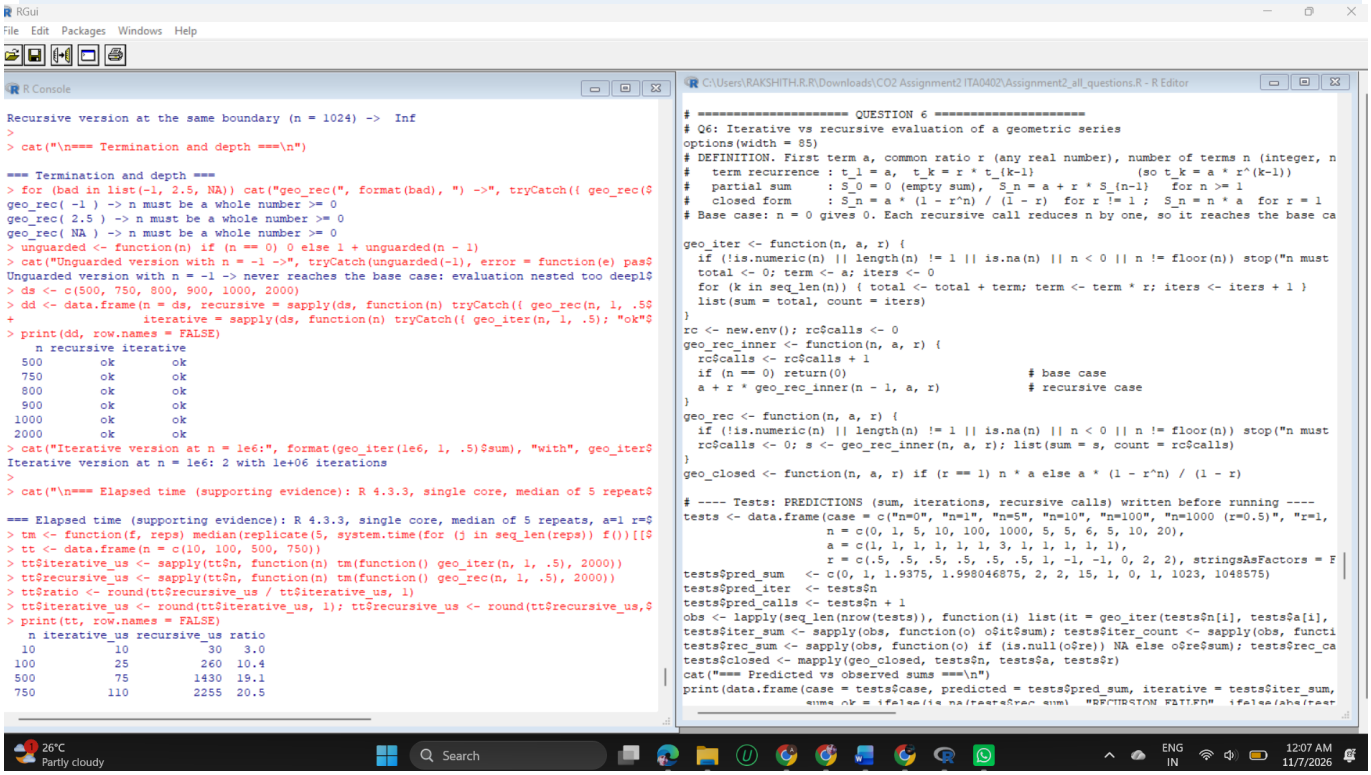
```
  n iterations rec_calls    sum_iter     sum_rec abs_diff
  0          0         1 0.000000000 0.000000000        0
  1          1         2 1.000000000 1.000000000        0
 10         10        11 1.998046875 1.998046875        0
100        100       101 2.000000000 2.000000000        0
500        500       501 2.000000000 2.000000000        0
750        750       751 2.000000000 2.000000000        0

=== Ratio boundary cases ===
r = 1 : closed form (1-r^n)/(1-r) would be 0/0, so the n*a branch gives 5 (handled by the n*a branch); iterative n=5, a=3 gives 15
r = -1: alternating partial sums 1,0,1,0 -> n=5: 1  n=6: 0  (no convergence, no error)
|r| < 1 converges to a/(1-r) = 2 for r = 0.5; S_100 = 2
r = 2 overflow boundary, S_n for n = 1023, 1024, 1025: 8.988e+307        Inf        Inf (n = 1024 overflows double precision to Inf)
Recursive version at the same boundary (n = 1024) ->  ERROR: recursion limit reached before the numbers overflow

=== Termination and depth ===
geo_rec( -1 ) -> n must be a whole number >= 0
geo_rec( 2.5 ) -> n must be a whole number >= 0
geo_rec( NA ) -> n must be a whole number >= 0
Unguarded version with n = -1 -> never reaches the base case: C stack usage  7969860 is too close to the li
    n          recursive iterative
  500                ok        ok
  750                ok        ok
  800                ok        ok
  900 FAILED (stack limit)        ok
 1000 FAILED (stack limit)        ok
 2000 FAILED (stack limit)        ok
Iterative version at n = 1e6: 2 with 1e+06 iterations

=== Elapsed time (supporting evidence): R 4.3.3, single core, median of 5 repeats, a=1 r=0.5 ===
  n iterative_us recursive_us ratio
 10         1.5         8.0   5.3
100         4.0        67.5  16.9
500        17.0       505.0  29.7
```

750 24.5 805.5 32.9



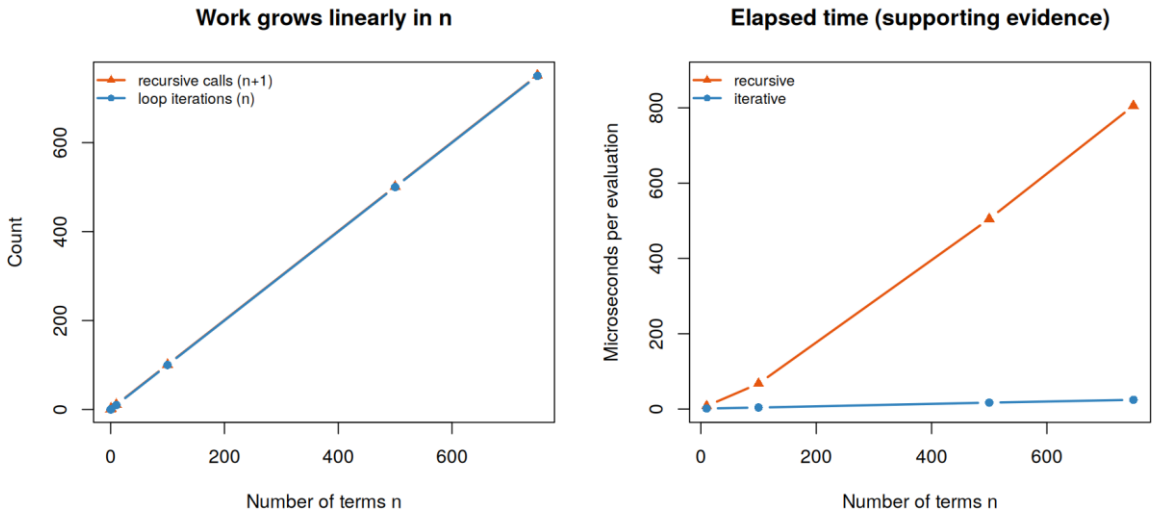


Figure 12: Left: iterations and recursive calls grow linearly. Right: elapsed time per evaluation (supporting evidence)

6.3 Analysis

- **Prediction versus observation:** 11 of 12 cases matched in sum, iterations and calls, which is 34 of 36 values. The exception is **n = 1000: the recursive version stops with a C stack error**, so its sum and call count do not exist (the iterative result did match). I had predicted that the recursion would return 2.
- **Linear cost:** iterations are exactly n and recursive calls exactly n + 1 for n = 0, 1, 10, 100, 500 and 750.

- **Agreement:** the iterative sum, the recursive sum and the closed form were identical in every test (largest difference 0).
- **Boundary ratios:** r = 1 makes the closed form 0/0, which the n x a branch avoids; r = -1 never converges but raises no error; r = 0 returns just a; for |r| < 1 the sums approach a/(1 - r) = 2. For r = 2 the sum overflows to Inf at n = 1024 in double precision, and the recursive version cannot even reach that n.
- **Termination guard:** n = -1, 2.5 and NA are rejected with a clear message. An unguarded recursive version given n = -1 never reaches the base case and ends with a stack error.
- **Depth limit:** recursion succeeded at n = 800 and failed at n = 900, 1000 and 2000; the iterative version also handled n = 1,000,000 (1,000,000 iterations). The exact limit depends on how much stack is already in use, so it should be treated as 'a few hundred', not as a fixed number.
- **Elapsed time (supporting only):** per evaluation the iterative version took 1.5 microseconds at n = 10 and 24.5 at n = 750; the recursive version took 8.0 and 805.5 (about 5 and 33 times slower).

6.4 Evaluation and recommendation

The iterative version is the better choice for real use: it has no depth limit, uses constant memory and was both faster and equally accurate. The recursive version mirrors the mathematical recurrence, which makes it useful for explaining the idea and for small n. The closed form is cheapest of all (one power and one division) but needs the r = 1 special case and can overflow in the same way.

QUESTION 7 - Diagnose, Redesign and Validate One Implementation (8 Marks)

7.1 Selected implementation

The implementation chosen is the **Question 5 file counter** (both the recursive version and the explicit-stack version). Question 5 had already shown a depth failure for recursion; this question reproduces it with a stated protocol, finds a second defect in the stack version, defines success criteria, redesigns the code and validates the redesign on equivalent inputs.

7.2 Diagnosis: two reproducible defects

- **Defect 1 (recursion, depth).** Ten trials per depth in the same calling context: depths 500, 700 and 800 succeeded 10 of 10 times, depths 900, 1000, 1500 and 3000 failed 10 of 10 times. Between 800 and 900 the outcome depends on the calling context and garbage collection. Error text: C stack usage 7970964 is too close to the limit. **Root cause:** every nesting level adds function-call frames on the C stack, and the 8 MB stack limit is reached long before memory runs out.
- **Defect 2 (stack version, wide folders).** One folder holding 5,000, 10,000, 20,000 and 40,000 files took 0.128, 0.387, 1.521 and 6.327 seconds: each doubling multiplied the time by 3.0 to 4.2, which is quadratic growth. The recursive version needed 0.003 to 0.028 s for the same folders. **Root cause:** stack[[length(stack)]] <- NULL builds a new, shorter list on every pop and copies the whole stack.

7.3 Evaluation criteria (fixed before redesigning)

ID	Criterion	Target
K1	Correctness	Same file count on 100% of the Q5 structures and on random trees
K2	Depth safety	Handles a chain of depth 100,000
K3	Operation count	Nodes visited equal the recursive call count
K4	Scalability on wide folders	Time grows by less than x3 when the folder size doubles
K5	Maintainability	At most 20 lines, base cases explicit, no packages

7.4 Redesign

The redesigned counter works **level by level**. It takes the whole current level as a vector, counts the files with vector tests, and joins the children of all folders in the level into the next level with unlist(recursive = FALSE). There is no per-node copying and the loop runs once per level.

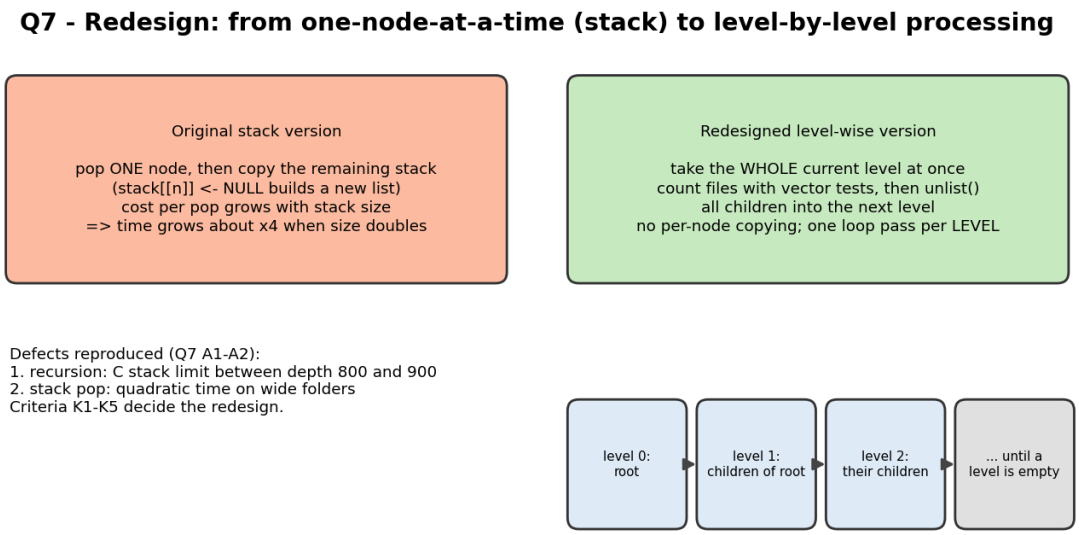


Figure 13: From one-node-at-a-time processing to level-by-level processing

Pseudocode

```
level <- [root]; files <- 0; visited <- 0
while level is not empty:
   visited <- visited + size(level)
   files   <- files + number of entries that are neither NULL nor a folder
   level   <- concatenation of the children of every folder in level
return files, visited
```

7.5 Implementation and validation

R code

```
# Q7: Diagnose - Redesign - Validate review of the Q5 file counter
options(width = 85)
# ================== A. DIAGNOSE (reuses count_files and count_files_stack from Q5) ==================
cat("=== A1. Defect 1: recursion fails on deep folders (10 trials per depth, same calling context) ===\n")
chain_of <- function(d) { x <- "leaf.txt"; for (i in seq_len(d)) x <- list(x); x }
rec_ok <- function(d) tryCatch({ calls$n <- 0; calls$max_depth <- 0; count_files(chain_of(d)); TRUE }, error = function(e) FALSE)
depths_a <- c(500, 700, 800, 900, 1000, 1500, 3000)
trial <- data.frame(chain_depth = depths_a, recursive_ok = sapply(depths_a, function(d) sum(replicate(10, rec_ok(d)))))
```

```
trial$result <- paste0(trial$recursive_ok, "/10 succeed")
print(trial[, c("chain_depth", "result")], row.names = FALSE)
always_fail <- min(depths_a[trial$recursive_ok == 0]); always_ok <- max(depths_a[trial$recursive_ok == 10 & depths_a < always_fail])
cat("Always succeeds up to depth", always_ok, "| always fails from depth", always_fail, "| unstable in between (depends on calling context and garbage collection)\n")
msg <- tryCatch(count_files(chain_of(always_fail)), error = function(e) conditionMessage(e))
cat("Error text at depth", always_fail, ":", substr(msg, 1, 60), "\n")
cat("Root cause: each nesting level adds R function-call frames on the C stack; the 8 MB limit is reached long before memory runs out.\n")

cat("\n=== A2. Defect 2: the stack version slows down quadratically on wide folders ===\n")
wide_of <- function(n) as.list(sprintf("f%d.txt", seq_len(n)))
sizes <- c(5000, 10000, 20000, 40000)
wd <- do.call(rbind, lapply(sizes, function(n) {
  tr <- wide_of(n)
  data.frame(children = n, stack_s = system.time(count_files_stack(tr))[["elapsed"]], recursive_s = system.time({ calls$n <- 0; count_files(tr) })[["elapsed"]])
}))
wd$stack_growth_vs_previous <- c(NA, round(wd$stack_s[-1] / pmax(wd$stack_s[-nrow(wd)], 0.001), 1))
print(wd, row.names = FALSE)
cat("Doubling the folder size multiplies the stack time by about 4 (quadratic); the recursive loop stays linear.\n")
cat("Root cause: stack[[length(stack)]] <- NULL builds a new, shorter list on every pop, copying the whole stack.\n")

# ================ B. EVALUATION CRITERIA ================
cat("\n=== B. Criteria (measurable) ===\n")
crit <- data.frame(id = paste0("K", 1:5), criterion = c("Correctness", "Depth safety", "Operation count", "Scalability on wide folders", "Maintainability"),
  target = c("Same file count on 100% of test and random trees", "Handles chain depth >= 100,000", "Nodes visited equal recursive calls",
             "Time grows less than x3 when size doubles", "Code at most 20 lines, base cases explicit"))
print(crit, right = FALSE, row.names = FALSE)

# ================ C. REDESIGN ================
cat("\n=== C. Redesign: process the tree LEVEL BY LEVEL (iterative, vectorised) ===\n")
```

```
cat("Pseudocode:\n  level <- [root]; files <- 0; visited <- 0\n  while level is not empty:\n    visited += size(level)\n  files   += count of entries that are neither NULL nor a folder\n    level    = concatenation of the children of every folder in level\n  return files, visited\n")
count_files_levels <- function(root) {
  level <- list(root); files <- 0; visited <- 0; passes <- 0
  while (length(level) > 0) {
    passes <- passes + 1; visited <- visited + length(level)
    is_null <- vapply(level, is.null, logical(1)); is_folder <- vapply(level, is.list, logical(1))
    files <- files + sum(!is_null & !is_folder)
    level <- unlist(level[is_folder], recursive = FALSE, use.names = FALSE)
  }
  list(files = files, visited = visited, passes = passes)
}
cat("Lines of code: recursive =", length(deparse(count_files)), "| stack =", length(deparse(count_files_stack)), "| level-wise =", length(deparse(count_files_levels)), "\n")
cat("Trade-off: the level-wise version loses the one-to-one match with the recursive definition and keeps a whole level in memory at once.\n")

# ================= D. VALIDATE with equivalent inputs =================
cat("\n=== D1. Correctness on the 7 Q5 structures and 300 random trees ===\n")
q5 <- sapply(tests, function(t) count_files_levels(t)$files); q5r <- sapply(tests, function(t) run_rec(t)$files)
cat("Q5 structures: level-wise equals recursive on", sum(q5 == q5r), "of", length(tests), "\n")
gen_tree <- function(depth) {
  if (depth == 0 || runif(1) < 0.25) return(if (runif(1) < 0.1) NULL else sprintf("f%d", sample(1e6, 1)))
  k <- sample(0:4, 1); lapply(seq_len(k), function(i) gen_tree(depth - 1))
}
set.seed(77); agree <- 0; same_visits <- 0; total_nodes <- 0
for (i in 1:300) {
  tr <- list(gen_tree(10), gen_tree(10), gen_tree(10))
  a <- run_rec(tr); b <- count_files_stack(tr); cc <- count_files_levels(tr)
  agree <- agree + (a$files == b$files && a$files == cc$files)
  same_visits <- same_visits + (a$calls == b$pops && a$calls == cc$visited)
  total_nodes <- total_nodes + a$calls
}
cat("Random trees (300, with empty folders and NULL links, ", total_nodes, " nodes in total):\n", sep = "")
cat("  all three methods give the same file count on", agree, "of 300\n")
cat("  recursive calls = stack pops = level-wise visits on", same_visits, "of 300\n")
```

```
cat("\n=== D2. Depth safety ===\n")
dd <- c(750, 1000, 20000, 100000)
dtab <- do.call(rbind, lapply(dd, function(d) { tr <- chain_of(d)
  data.frame(chain_depth = d, recursive = paste0(sum(replicate(5, rec_ok(d))), "/5 ok"), stack = tryCatch({
count_files_stack(tr)$files; "ok" }, error = function(e) "FAILED"),
           level_wise = tryCatch({ r <- count_files_levels(tr); paste0("ok (", r$files, " file, ", r$passes, " passes)") },
error = function(e) "FAILED")) }))
print(dtab, row.names = FALSE)

cat("\n=== D3. Wide folders (equivalent inputs; seconds) ===\n")
sizes2 <- c(5000, 10000, 20000, 40000, 160000)
wt <- do.call(rbind, lapply(sizes2, function(n) { tr <- wide_of(n)
  data.frame(children = n, recursive = system.time({ calls$n <- 0; count_files(tr) })[["elapsed"]],
           stack = if (n <= 40000) system.time(count_files_stack(tr))[["elapsed"]] else NA,
           level_wise = system.time(count_files_levels(tr))[["elapsed"]]) }))
print(wt, row.names = FALSE)
r160 <- count_files_levels(wide_of(160000))
cat("Level-wise result for 160,000 files:", r160$files, "files,", r160$visited, "nodes visited,", r160$passes,
"passes\n")

cat("\n=== D4. Complete trees (balanced, equivalent inputs; seconds, median of 3) ===\n")
tt <- do.call(rbind, lapply(c(6, 7, 8, 9), function(d) { tr <- make_tree(d, 4)
  data.frame(depth = d, nodes = (4^(d + 1) - 1) / 3,
    recursive = median(replicate(3, system.time({ calls$n <- 0; count_files(tr) })[["elapsed"]])),
    stack = median(replicate(3, system.time(count_files_stack(tr))[["elapsed"]])),
    level_wise = median(replicate(3, system.time(count_files_levels(tr))[["elapsed"]]))) }))
print(tt, row.names = FALSE)

cat("\n=== Scorecard against the criteria (computed from the measurements above) ===\n")
growth <- function(col) wt[[col]][4] / wt[[col]][3]            # time at 40,000 / time at 20,000 children
lines_of <- c(rec = length(deparse(count_files)), stack = length(deparse(count_files_stack)), level =
length(deparse(count_files_levels)))
verdict <- function(ok) ifelse(ok, "PASS", "FAIL")
sc <- data.frame(id = crit$id, criterion = crit$criterion,
  recursive = c(verdict(agree == 300), verdict(grepl("^5/5", dtab$recursive[4])), verdict(same_visits == 300),
verdict(growth("recursive") < 3), verdict(lines_of["rec"] <= 20)),
```

```
 naive_stack = c(verdict(agree == 300), verdict(dtab$stack[4] == "ok"), verdict(same_visits == 300), verdict(growth("stack") < 3), verdict(lines_of["stack"] <= 20)),
 level_wise = c(verdict(agree == 300 && all(q5 == q5r)), verdict(grepl("ok", dtab$level_wise[4])), verdict(same_visits == 300), verdict(growth("level_wise") < 3), verdict(lines_of["level"] <= 20)))
print(sc, right = FALSE, row.names = FALSE)
cat("Growth factor when the folder doubles from 20,000 to 40,000 children: recursive", round(growth("recursive"), 2), "| stack", round(growth("stack"), 2), "| level-wise", round(growth("level_wise"), 2), "\n")
cat("Lines of code: recursive", lines_of["rec"], "| stack", lines_of["stack"], "| level-wise", lines_of["level"], "\n")

png("fig/fig_q7_redesign.png", width = 1800, height = 850, res = 170)
par(mfrow = c(1, 2), mar = c(5, 5, 4, 1))
plot(wt$children, wt$recursive, type = "b", pch = 16, col = "#3182bd", lwd = 2, log = "xy", xlab = "Children in one folder", ylab = "Seconds (log scale)",
    main = "Wide folders: time vs size", ylim = c(0.0005, 10))
lines(wt$children, wt$stack, type = "b", pch = 15, col = "#e6550d", lwd = 2)
lines(wt$children, wt$level_wise, type = "b", pch = 17, col = "#31a354", lwd = 2)
legend("topleft", legend = c("naive stack", "recursive", "level-wise"), col = c("#e6550d", "#3182bd", "#31a354"), pch = c(15, 16, 17), lwd = 2, bty = "n", cex = 0.8)
barplot(c(recursive = always_ok, stack = 100000, level_wise = 100000), log = "y", ylim = c(100, 300000), col = c("#3182bd", "#e6550d", "#31a354"),
      ylab = "Deepest chain handled (log scale)", main = "Depth safety (tested up to 100,000)", cex.names = 0.8)
dev.off()
```

Output (captured from running the code above)

```
=== A1. Defect 1: recursion fails on deep folders (10 trials per depth, same calling context) ===
 chain_depth        result
         500 10/10 succeed
         700 10/10 succeed
         800 10/10 succeed
         900  0/10 succeed
        1000  0/10 succeed
        1500  0/10 succeed
        3000  0/10 succeed
Always succeeds up to depth 800 | always fails from depth 900 | unstable in between (depends on calling context and garbage collection)
Error text at depth 900 : C stack usage  7970964 is too close to the limit
```

Root cause: each nesting level adds R function-call frames on the C stack; the 8 MB limit is reached long before memory runs out.

=== A2. Defect 2: the stack version slows down quadratically on wide folders ===

children	stack_s	recursive_s	stack_growth_vs_previous
5000	0.128	0.003	NA
10000	0.387	0.006	3.0
20000	1.521	0.015	3.9
40000	6.327	0.028	4.2

Doubling the folder size multiplies the stack time by about 4 (quadratic); the recursive loop stays linear.
Root cause: stack[[length(stack)]] <- NULL builds a new, shorter list on every pop, copying the whole stack.

=== B. Criteria (measurable) ===

id	criterion	target
K1	Correctness	Same file count on 100% of test and random trees
K2	Depth safety	Handles chain depth >= 100,000
K3	Operation count	Nodes visited equal recursive calls
K4	Scalability on wide folders	Time grows less than x3 when size doubles
K5	Maintainability	Code at most 20 lines, base cases explicit

=== C. Redesign: process the tree LEVEL BY LEVEL (iterative, vectorised) ===
Pseudocode:

```
level <- [root]; files <- 0; visited <- 0
while level is not empty:
  visited += size(level)
  files   += count of entries that are neither NULL nor a folder
  level    = concatenation of the children of every folder in level
return files, visited
```

Lines of code: recursive = 16 | stack = 18 | level-wise = 17
Trade-off: the level-wise version loses the one-to-one match with the recursive definition and keeps a whole level in memory at once.

=== D1. Correctness on the 7 Q5 structures and 300 random trees ===
Q5 structures: level-wise equals recursive on 7 of 7
Random trees (300, with empty folders and NULL links, 164715 nodes in total):
all three methods give the same file count on 300 of 300
recursive calls = stack pops = level-wise visits on 300 of 300

```
=== D2. Depth safety ===
 chain_depth recursive stack                level_wise
         750    5/5 ok    ok    ok (1 file, 751 passes)
        1000    0/5 ok    ok   ok (1 file, 1001 passes)
       20000    0/5 ok    ok  ok (1 file, 20001 passes)
      100000    0/5 ok    ok ok (1 file, 100001 passes)

=== D3. Wide folders (equivalent inputs; seconds) ===
 children recursive stack level_wise
     5000     0.004 0.113      0.002
    10000     0.007 0.410      0.003
    20000     0.015 1.601      0.005
    40000     0.028 6.540      0.011
   160000     0.119    NA      0.050
Level-wise result for 160,000 files: 160000 files, 160001 nodes visited, 2 passes

=== D4. Complete trees (balanced, equivalent inputs; seconds, median of 3) ===
 depth  nodes recursive stack level_wise
     6   5461     0.004 0.003      0.002
     7  21845     0.015 0.011      0.006
     8  87381     0.060 0.047      0.023
     9 349525     0.254 0.197      0.093

=== Scorecard against the criteria (computed from the measurements above) ===
 id criterion                 recursive naive_stack level_wise
 K1 Correctness                  PASS      PASS        PASS
 K2 Depth safety                 FAIL      PASS        PASS
 K3 Operation count              PASS      PASS        PASS
 K4 Scalability on wide folders PASS       FAIL        PASS
 K5 Maintainability              PASS      PASS        PASS
Growth factor when the folder doubles from 20,000 to 40,000 children: recursive 1.87 | stack 4.08 | level-wise
2.2
Lines of code: recursive 16 | stack 18 | level-wise 17
```

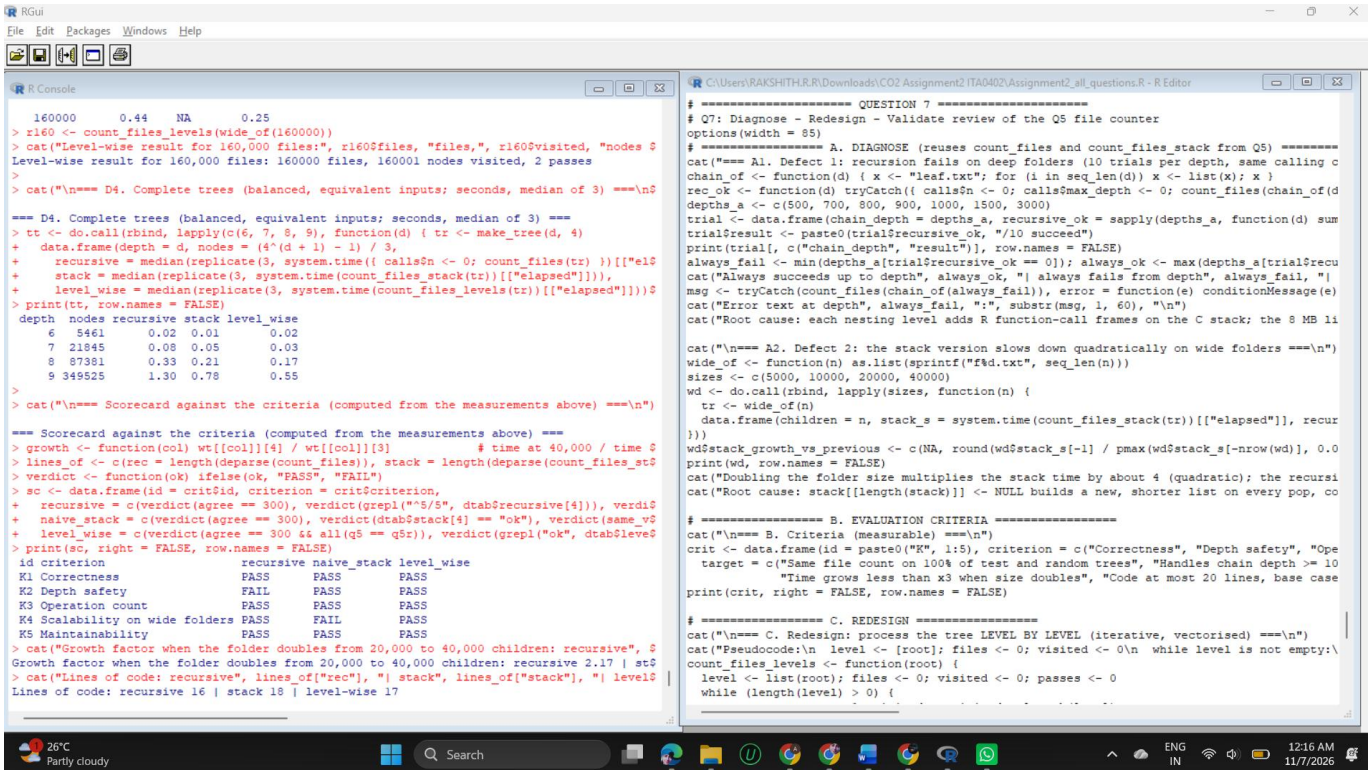


Figure 14: Left: time against folder size (log scales). Right: deepest chain each version handled

7.6 Results and defence

Criterion	Recursive	Naive stack	Level-wise	Evidence
K1 Correctness	PASS	PASS	PASS	Equal file counts on the 7 Q5 structures and on 300 random trees (164,715 nodes, with empty folders and NULL links)

K2 Depth safety	FAIL	PASS	PASS	Chains of 1,000, 20,000 and 100,000: recursion failed 5 of 5; stack and level-wise succeeded (100,001 passes for the longest chain)
K3 Operation count	PASS	PASS	PASS	Calls = pops = visits on 300 of 300 random trees
K4 Wide folders	PASS	FAIL	PASS	Growth when 20,000 doubles to 40,000: recursive 1.87, stack 4.08, level-wise 2.20
K5 Maintainability	PASS	PASS	PASS	16, 18 and 17 lines of code

- **The redesign meets all five criteria; neither alternative does.** For a folder of 160,000 files the level-wise version needed 0.050 s and two passes; the naive stack version would need several minutes at that size by the measured quadratic trend, so it was not run.
- **On balanced trees all three methods are comparable** (87,381 nodes: 0.060, 0.047 and 0.023 s), so the redesign does not slow ordinary cases.
- **Passes are not calls:** the level-wise loop runs once per level (751 passes for a chain of depth 750) while counting the same 751 visited nodes, which is why K3 compares visits, not loop passes.

7.7 Critical judgement and remaining weaknesses

- **The recursive version is not obsolete.** It is the clearest code, it passed every criterion except depth, and it is the natural choice when each subfolder must return its own value (per-folder subtotals), which the level-wise version would need extra bookkeeping to provide.
- **Cost of the redesign:** it no longer mirrors the recursive definition, and it holds a whole level in memory at once, so an extremely wide level costs memory proportional to its width.
- **What the evidence does not show:** the depth limit was measured on one machine and R version (4.3.3) and depends on the calling context; the timings are single measurements. A different R build could move the thresholds but not the quadratic growth of the stack version.
- **Recommendation:** use the level-wise counter for unknown or generated archives, and keep the recursive counter for small, known structures where clarity matters.

APPENDIX A - Submission Checklist

Required item	Where it is
Report	This document (sheet first, then Questions 1 to 7)
Source code	Code listing in each section, and the file Assignment2_all_questions.R
Datasets	Defined in each program; exported as CSV in raw_data_assignment2.zip
Raw experimental data and outputs	Output block in each section, and the CSV files (Q1 28 and 25,000 zones with predictions and observations, Q2 profiles and traces, Q3 datasets, Q4 sequences, counts and simulation)
Prediction versus observation	Section analysis of each question and the prediction scorecard on the overview page
Tables and graphs	14 figures: 7 design diagrams and 7 result graphs
Counts of iterations, calls, evaluations	Q1 sections 1.4-1.5, Q2 traces, Q3 session logs, Q4 section 4.4, Q5 and Q6 call counts, Q7 passes and visits
Pseudocode	Q7 section 7.4 (level-wise counter); the rule lists and flow diagrams of Q1 to Q4 serve as design notes

CSV files in raw_data_assignment2.zip

File	Content
q1_adversarial_zones_28.csv	28 zones, hand predictions, observed classes, average score, severe count
q1_changed_condition_25000_zones.csv	25,000 zones with v1 and v2 classes and condition evaluations
q2_application_profiles_12.csv	12 profiles with v1 and v2 status, reason and trace
q3_order_datasets.csv	the seven order-amount datasets
q4_reading_sequences.csv	the reading sequences (an empty sequence has no rows)
q4_counts_per_sequence.csv, q4_simulation_summary.csv	attempts, loop passes, evaluations; simulation by invalid-rate

Viva quick reference

- Q1: why is the override tested first, and what is its cost? (Safety; 47% more condition evaluations per zone.)
- Q2: why can documents never be waived, and how was the policy changed? (Named-condition waivers; one line in the policy table.)
- Q3: why is the registry menu easier to extend than switch()? (1 edit instead of 3 per operation.)

- Q4: when is the redesigned check order worse? (When more than about 67% of readings are invalid.)
- Q5 and Q6: what limits recursion in R? (The C stack: failure between depth 800 and 900 here.)
- Q7: which defects were reproduced and what fixed them? (Recursion depth and quadratic pops; level-by-level processing.)